COMPUTER SCIENCE + COMPUTER ENGINEERING

Foundations and Your First Game

Student handbook / lessons 01.1-05.4

From understanding one state change to building, testing and handing off a complete small game. Worked code, practical tasks and short quizzes follow every lesson.

20 expanded lessons · 20 runnable examples · 100 lesson questions

Package 02 · Expanded teaching edition · September 28, 2026
Volume 01 of the full undergraduate-to-graduate program
Self-study material. Completion requires implementation, testing and review.

Contents

Use the linked contents or PDF bookmarks to navigate.

Student orientation 3

Module 01: Starting as an engineer 4
- Lesson 01.1: What a program actually does 4
- Lesson 01.2: Files, terminals and reproducible execution 7
- Lesson 01.3: Trace before running 10
- Lesson 01.4: Units, budgets and engineering notebooks 13

Module 02: Values, decisions and repetition 16
- Lesson 02.1: Types and expressions 16
- Lesson 02.2: Conditions and boundary cases 19
- Lesson 02.3: Loops and invariants 22
- Lesson 02.4: Functions and contracts 25

Module 03: Organizing data and small programs 28
- Lesson 03.1: Sequences, dictionaries and identity 28
- Lesson 03.2: Decomposition, modules and interfaces 31
- Lesson 03.3: Files, serialization and validation 34
- Lesson 03.4: Classes, composition and state machines 37

Module 04: Debugging, testing and collaboration 40
- Lesson 04.1: Debugging as an experiment 40
- Lesson 04.2: Unit, integration and property tests 43
- Lesson 04.3: Git, branches and recoverable changes 46
- Lesson 04.4: Review, documentation and responsible AI help 49

Module 05: A complete first game 52
- Lesson 05.1: The update loop and command model 52
- Lesson 05.2: Collision rules and complete behavior 55
- Lesson 05.3: Deterministic replay and simple measurement 58
- Lesson 05.4: Finishing, usability and the first portfolio artifact 61

Primary-source reading directory 64

Student orientation

This is Package 02, the expanded first teaching volume in the ten-volume program. It preserves the original module/lesson IDs and learning outcomes while replacing short primers with fuller teaching, examples, practical instructions and application checks. Begin with START_NEXT.pdf for setup and the continuation point.

How to use each lesson: retrieve the prerequisite, make a prediction, read the explanation, run the example, complete the lab, solve the changed case and answer the quiz before consulting the separate key. Questions 1-3 are retained core questions; 4-5 are new application questions. Keep the core 5/6 threshold and achieve at least 8/10 overall, with study and practical evidence complete. References in square brackets identify the primary-source directory at the back; they support language/tool semantics, not a promise that reading alone replaces the lab.

The first-module function starters are optional return exercises after 02.4. Labs 01.2, 04.3, 04.4 and 05.4 are evidence-based rather than automatically scored functions. Supplied reference tests check supplied reference code; the learner's code and project require their own checks. Keep ANSWER_KEY.pdf closed until an independent attempt.

The language examples are intentionally small and use only the Python standard library. Code excerpts in the worked-example sections are also supplied verbatim as runnable files. Demonstrations labeled as buggy/review candidates are intentionally incorrect teaching cases, not recommended implementations. The software targets Python 3.11+ and was executed in Python 3.13.5 on Linux; other platforms are not claimed as physically tested.

This volume leads to an independently built first game, not an accredited credential. The original Volume 1 planning allowance is 220-340 hours including practice and projects; the added teaching package elaborates that same work rather than creating a second additive allowance.

Module 01: Starting as an engineer

Prerequisites: None. Basic reading, arithmetic and the ability to save a file.

Integration project: An explainable motion experiment. The full brief and review are in the Lab Workbook.

Lesson 01.1: What a program actually does

Learning outcome: Explain state, instructions, input and output using a tiny game.

The four parts of a tiny game

Imagine a maintenance robot in a training yard. It has a location, some energy and a destination. A player asks it to move right. Before writing any code, separate four things: the **input** is the request; the **state** is the stored location and energy; the **rule** decides what the request is allowed to change; the **output** tells the player what happened. A rule may reject a request. Rejection is still a defined result, not the absence of programming.

Suppose the robot starts at location 2 and receives a request to move right by 3 units. The rule produces location 5. Showing `Robot at 5` on a terminal does not create a second movement. Neither does showing the same state in a second window. This distinction is the first version of an engine boundary: simulation changes the world; presentation observes it. It will later make headless tests, replays, multiple views and dedicated servers possible within designs that preserve the boundary.

A useful rule can be written as `next state = update(current state, command)`. This is a design model, not Python syntax. Its value is that it forces us to name the information a decision needs. When a program gives an unexpected answer, ask which input, state value or rule differed from the model.

What information must persist?

A robot position must survive between commands. A message such as `Moved right` may be temporary. A movement speed is a configuration value unless the game changes it during play. A wall map is also information; even when it is constant, a collision rule depends on it. Do not call every variable state without explaining its lifetime and purpose.

Consider an inventory display reading `3 keys`. The text can be regenerated from the number 3. Storing both the number and the text as independently editable values creates a consistency problem: which is correct after a key is spent? For our first game, store the underlying facts and derive the display. Later systems sometimes cache derived values for performance, but then they must define when the cache becomes invalid.

Source code is not the screen

Source code is text written for a programming language. In this course a Python interpreter runs that text; the operating system provides services such as files and terminal access. Hardware executes the underlying instructions. An engine is another layer of software, not a special kind of processor. Keep a simple layered explanation in your notebook: game rules, reusable engine facilities, operating-system services, processor and memory.

You do not need to understand instruction decoding to begin. You do need to avoid explanations such as “the computer knew what I wanted.” A program acts according to the instructions and inputs it actually receives. A compiler or interpreter can detect some mistakes, but it does not know whether you intended a closed door or an open one.

Predict a complete transition

Make a table with columns for step, command, previous position, change and next position. Record the initial position as step zero. Do not skip a zero-distance command: its unchanged result is useful evidence that doing nothing is a legitimate case. When your final position happens to match the expected value, still check intermediate positions. Two wrong updates can cancel and produce a misleading final answer.

Our first exercise is deliberately small enough to solve on paper. A prediction is more valuable than an attractive interface at this stage. Keep the paper trace even after the program agrees with it; it becomes an independent expected result rather than an answer copied from the implementation.

Worked example

Predict before running `examples/example_01_1.py`.

```
position = 2
print("start:", position)
position = position + 3
print("after right 3:", position)
position = position - 1
print("after left 1:", position)
print("draw again:", position)
```

The first assignment establishes a fact; the first print only reports it. The next two assignments change that fact in sequence. The last print reads the same value as the preceding print, because no rule has changed the position between them. Predict four displayed values before running. Replacing the last print with `position = position + 1` would be a rule change, not a harmless drawing change.

Expected demonstration output

```
start: 2
after right 3: 5
after left 1: 4
draw again: 4
```

Practical work and pass evidence

Start on paper at x=2. Trace right 3, left 1, right 0, left 4, right 2. Include the initial state and all five following states. Mark which operations are reads and which are writes. Then run `examples/example_01_1.py` twice and compare its smaller demonstration with your prediction.

Create `workspace/01_1_trace.md`. Include the six-position trace, a plain-language update rule, and an example where two wrong movements cancel. Your pass evidence is an explanation of every transition and why drawing twice must not move twice. The optional function scaffold `labs/student/lab_01_1.py` is a return visit after Lesson 02.4; functions and loops are not prerequisites for this paper-first lesson.

Changed-case challenge

Change the initial location to -3 without changing the commands. Predict every state, not only the final value.

Companion source study

[S01] Harvard CS50 Introduction to Programming with Python; [S02] Python: An Informal Introduction. Read the sections relevant to the named concepts; record the section studied and one additional example.

Five-question lesson quiz

Answer independently. Core questions 1-3: at least 5/6. All five: at least 8/10, plus practical evidence.

1. What is the difference between state and a displayed image?
2. What final position results from the five lab commands?
3. Why should drawing a character not update its position?
4. A game stores health=10 and a label reading "10". Damage changes health to 7 but not the label. Which information should normally be authoritative in this first game, and how would you prevent disagreement?
5. Two implementations finish at x=2. Is that sufficient to show they applied the five commands correctly?

Evidence to retain: predictions, code or worksheet, actual output, changed-case reasoning, quiz answers and corrections.

Lesson 01.2: Files, terminals and reproducible execution

Learning outcome: Run a saved Python program and distinguish a shell from an interpreter.

Four objects that are easy to confuse

A **file** stores bytes. A **folder** organizes file names. A **terminal** is an interface for text interaction. A **shell** is the program interpreting shell commands in that interface. Python is another program the shell can launch. A terminal window may therefore show a shell prompt at one moment and a Python prompt at another. The visible window alone does not identify which language currently accepts your input.

A command such as `python3 examples/example_01_1.py` belongs in the shell. A statement such as `print(2 + 3)` belongs in Python code or at a Python prompt. Do not copy a prompt marker such as `$`, `PS>` or `>>>` into the command itself. The marker is notation showing where the command is entered, not part of what you type.

Locate the package, then locate the program

Extract the ZIP before executing its files. Open a terminal at the extracted `Game_Engineering_Package_02` folder. Its immediate children include `examples`, `tools`, `labs` and `capstone`. The working directory should be that folder, not its parent and not an unrelated downloads folder. A relative path such as `examples/example_01_1.py` is interpreted from the working directory.

On macOS/Linux, use `pwd` to see the directory and `ls` to inspect it. In PowerShell, use `Get-Location` and `Get-ChildItem`. To change directories, use `cd` followed by the real folder path. Quote a path that contains spaces. Do not substitute a fictional user name from someone else's example into your own filesystem.

Run `python3 --version` on macOS/Linux or `py -3 --version` on Windows. A working `python --version` that reports Python 3 can be used instead. Use the same verified launcher throughout the package. The examples target Python 3.11 or later and were executed here with Python 3.13.5; newer-version and Windows/macOS execution are not claimed as completed tests. No third-party Python package is required for the supplied runtime examples.

Read an error as evidence

A missing interpreter error comes from the shell or launcher before your script begins. A “cannot open file” message may come from Python because the interpreter ran but could not find the script. A syntax error means Python found source text but could not parse it. An exception during execution means it began running and encountered an unsupported operation or explicit failure.

These are different layers. Editing the movement formula cannot repair a misspelled filename. Reinstalling Python is not the first response to a wrong output value. Identify the layer, reproduce the failure once and change the smallest relevant thing. In this lesson, a deliberately misspelled example filename is a safe failure experiment because it does not modify files.

Record an executable recipe

A reproducibility note should name the source file or revision, interpreter version, working directory, exact command, inputs, expected output and actual output. A screenshot can supplement the record, but plain text is easier to search and copy. Avoid recording private paths or identifiers in a public portfolio without review.

Run `tools/environment_check.py` for a read-only report. It does not install software or change settings. A missing Git installation is not a blocker until the Git lesson. When software installation is needed, follow the official setup documentation and the device owner's approved process. Nothing in this volume requires administrator commands, changing security settings, a paid cloud account or an engine installation.

Worked example

Predict before running `examples/example_01_2.py`.

```
from pathlib import Path
import sys

print("python:", sys.version.split()[0])
print("script name:", Path(__file__).name)
print("working directory name:", Path.cwd().name)
print("script exists:", Path(__file__).is_file())
```

`__file__` identifies this script's file. `Path.cwd()` asks for the process working directory; it is not necessarily the directory containing the script. The two may coincide, but this example intentionally reports them separately. Version and directory-name output are environment-dependent, so compare their meaning rather than expecting another learner's exact text.

Expected demonstration output

```
python: [your actual Python version]
script name: example_01_2.py
working directory name: [your current folder name]
script exists: True
```

Practical work and pass evidence

Run `tools/environment_check.py` and save the output with your actual command in `workspace/01_2_environment.md`. Run the movement example, then deliberately change only its filename in the command to a name that does not exist. Record the resulting error, identify which program emitted it and repair the command.

Repeat the valid command from the package root. Your evidence must distinguish a script path from the working directory and explain how a relative filename is resolved. This is an evidence-based lab: no Python function starter is assigned and the automatic lab checker does not certify environment understanding.

Changed-case challenge

Run the example using its absolute path from a different working directory. Predict which reported field changes before executing.

Companion source study

[S02] Python: An Informal Introduction; [S03] Python Setup and Usage. Read the sections relevant to the named concepts; record the section studied and one additional example.

Five-question lesson quiz

Answer independently. Core questions 1-3: at least 5/6. All five: at least 8/10, plus practical evidence.

1. What does a relative filename depend on?
2. Is cd a Python statement in an ordinary script?
3. What three facts help another person reproduce a run?
4. You are in the package root, but type `python3 example_01_1.py`. The file is inside examples/. What is the smallest likely repair?
5. A saved edit produces no change in output. Give two checks before reinstalling anything.

Evidence to retain: predictions, code or worksheet, actual output, changed-case reasoning, quiz answers and corrections.

Lesson 01.3: Trace before running

Learning outcome: Predict a program by following assignments in order.

Assignment is an event, not a permanent equation

In mathematics, an equation often states a relationship to be solved or maintained. A Python assignment performs an operation at a particular moment. The right side is evaluated using the values available then, and the resulting object is bound to the name on the left. Later assignments do not automatically rerun earlier expressions.

With `health = 10` followed by `after_hit = health - 3`, the second name refers to the result 7. Setting `health = 20` afterward does not change `after_hit` to 17. That would require another evaluation. A display formula recalculated each frame is a separate operation; it is not a magical property of assignment.

Trace one statement at a time

Use a row after every statement that changes a name. Before a name has been assigned, mark it as “not yet defined” rather than guessing zero. A program that reads such a local name may fail; names are not automatically initialized to whatever would make the algorithm work.

For each row, write the old values needed by the right side, evaluate the expression, then update the left side. This discipline prevents a common mistake: mentally using a future value too early. It also shows why changing statement order can change behavior. “These are the same lines” is not a correctness argument when they run in a different sequence.

The overwritten-value problem

Suppose two players have scores 3 and 8 and you want to exchange them. Writing `left = right` first makes `left` refer to 8. A later `right = left` reads that new 8, not the original 3. The old left value is no longer accessible through either name. The algorithm lost information before it used it.

A temporary name preserves the old value until both assignments are complete. Python also supports multiple assignment such as `left, right = right, left`; the right-hand values are evaluated before the new bindings are installed. Use the longer temporary-variable version first so that you can explain what the shorter version accomplishes. A concise expression is useful only when its semantics are understood.

Names and values are not always independent copies

For the integer examples here, rebinding one name does not modify the immutable integer referred to by another name. Do not generalize that to every object. Later, two names can refer to the same mutable list; changing the list through one name is then observable through the other. The right mental model is “names refer to objects,” not “every assignment makes a deep copy.” Lesson 03.1 develops the distinction with concrete collections.

This limitation matters even now. A trace table should record values and, when relevant, whether two names refer to the same object. For now use integers and strings; the goal is to master sequencing before adding shared mutable structures.

Predictions that can be wrong are useful

Write the expected output before you run the program. When the result disagrees, locate the first disagreeing row rather than only the last line. Then state a specific correction to your mental model. “Python is weird” does not identify a reusable lesson; “I expected an earlier expression to reevaluate” does.

Build a habit of separating a prediction from an observation in your notebook. Do not edit the prediction after seeing the result. The difference is the evidence showing what you learned. Over time this record will make recurring assumptions visible and make debugging more systematic.

Worked example

Predict before running `examples/example_01_3.py`.

```
left = 3
right = 8
saved_left = left
left = right
right = saved_left
print("swapped:", left, right)

x = 4
y = x + 2
x = 10
print("later values:", x, y)
```

`saved_left` keeps 3 accessible while `left` is rebound to 8. `right` then receives the preserved value. In the second sequence, `y` is calculated when `x` is 4. The assignment of 10 to `x` does not execute the `y` expression again. Write both traces before running the file.

Expected demonstration output

```
swapped: 8 3
later values: 10 6
```

Practical work and pass evidence

Create three four-statement assignment sequences in `workspace/01_3_trace.md`. Include an incorrect swap, the repaired swap and a sequence in which a derived value does not update automatically. For each, predict every binding and final printed output before checking with Python.

The optional starter `labs/student/lab_01_3.py` packages the swap in a function; return to it after Lesson 02.4. Pass now by explaining where the incorrect swap loses the old value and by solving a fresh sequence without running it first. Do not count a memorized one-line swap as sufficient explanation.

Changed-case challenge

Use starting values -5 and 0 in your repaired swap. Then make a two-variable example whose behavior changes when two assignments are reversed.

Companion source study

[S02] Python: An Informal Introduction; [S04] Python Built-in Types; [S09] Python: Classes. Read the sections relevant to the named concepts; record the section studied and one additional example.

Five-question lesson quiz

Answer independently. Core questions 1-3: at least 5/6. All five: at least 8/10, plus practical evidence.

1. After a = 3; b = a; a = 8, what is b for these integers?
2. Why does x = y; y = x fail to swap two different values?
3. What makes prediction useful when a run disagrees?
4. Trace `a=2; b=a*4; a=b-1; b=a+2`. What are a and b at the end?
5. Why should you preserve your incorrect prediction instead of rewriting it to match the output?

Evidence to retain: predictions, code or worksheet, actual output, changed-case reasoning, quiz answers and corrections.

Lesson 01.4: Units, budgets and engineering notebooks

Learning outcome: Use units and simple arithmetic to check game calculations.

Units make arithmetic interpretable

The expression `6 * 0.25` produces a number, but its meaning depends on what the inputs represent. Six units per second multiplied by one quarter of a second gives 1.5 units. Seconds cancel. Six pixels per frame multiplied by seconds does not have the same meaning. Naming variables `speed_units_per_second` and `dt_seconds` keeps that distinction visible.

Write the relationship before substituting numbers: displacement equals speed multiplied by elapsed time. Then state assumptions. Here speed is constant during the update and the model is one-dimensional. We are not yet modeling acceleration, collision or curved motion. A correct calculation under a simple model should not be described as a complete physical simulation.

Convert at the boundary

One second is 1,000 milliseconds. When a timing source reports 250 milliseconds, convert it to 0.25 seconds before using a speed expressed per second. Multiplying 6 by 250 would produce a displacement 1,000 times too large. Adding a position to a time is a different failure: the units do not describe a meaningful sum at all.

Prefer one internal unit convention. For this volume, elapsed time is in seconds and positions use explicitly named game units. Convert input into that convention once and format output separately. Do not scatter unexplained factors of 1,000 through unrelated functions; each is another place to invert a conversion accidentally.

A frame interval is a budget, not a measurement

For a target rate of f frames per second, a evenly spaced frame interval is `1/f` seconds or `1000/f` milliseconds. At 60 frames per second that is approximately 16.67 milliseconds. It is a planning interval, not evidence that a particular computer can draw a particular scene at that rate. A 20-millisecond operation alone cannot fit inside a 16.67-millisecond serial budget.

Be careful when adding work times. A first serial model might add input, simulation and drawing durations, but later CPU/GPU overlap requires dependency reasoning. We will not infer actual frame performance from this arithmetic. For now, the exercise teaches whether a stated target and a stated cost are even compatible under the explicitly chosen serial model.

Zero, sign and non-finite values

A negative velocity can legitimately mean movement left. A negative elapsed interval is excluded by this lesson's contract because the update is not a time-reversal operation. A zero interval is allowed and produces no movement. A target frame rate of zero cannot be converted to an ordinary finite frame interval by division.

Computers also have representations such as positive infinity and not-a-number. Later code uses `math.isfinite` to reject them at the boundary of calculations that require finite values. This does not replace unit reasoning: 250 milliseconds mislabeled as seconds is perfectly finite but wrong. Validation must match both representation and meaning.

Keep hypotheses separate from results

Use three headings in a laboratory note: prediction, observation and explanation. “I predict 15 units because 6 units/second times 2.5 seconds equals 15 units” is a prediction. “The program printed 15000” is an observation. “The input remained in milliseconds” is an explanation to test, not a fact established by the surprising output alone.

When recording measured performance later, preserve the workload, environment, trial method and raw samples. Here, record the arithmetic and chosen assumptions. Do not invent an execution-time result to fill a table. An unanswered measurement question is more useful than a precise-looking fictional number.

Worked example

Predict before running `examples/example_01_4.py`.

```
speed_units_per_second = 6.0
dt_milliseconds = 250.0
dt_seconds = dt_milliseconds / 1000.0
travel_units = speed_units_per_second * dt_seconds
print("travel units:", travel_units)

frames_per_second = 120
interval_ms = 1000.0 / frames_per_second
print("target interval ms:", round(interval_ms, 3))
```

The conversion occurs before the movement calculation. The frame result is rounded only for presentation; that does not improve the underlying timing behavior of a machine. Explain why multiplying milliseconds by 1,000 instead of dividing would create a second, much larger error. The example measures no hardware performance.

Expected demonstration output

```
travel units: 1.5
target interval ms: 8.333
```

Practical work and pass evidence

Calculate all six combinations of speeds 2, 6 and 10 units/second with intervals 0.1 and 0.5 seconds. Then calculate the frame intervals for 30, 60 and 120 frames/second. Show units and identify the chosen targets. Store the table in `workspace/01_4_units.md`.

After functions are introduced, implement the `displacement` and `frame_interval_ms` starter contracts. Their input domain is finite numeric values, with nonnegative elapsed time and positive frame rate; finite overflow results must also be rejected. These programmatic checks supplement rather than replace the written unit analysis.

Changed-case challenge

A movement program receives 16 milliseconds and a speed of 2.5 meters/second. Derive the displacement and compare it with the mistaken calculation using 16 seconds.

Companion source study

[S02] Python: An Informal Introduction; [S05] Python: Floating-Point Arithmetic; [S14] Python math library. Read the sections relevant to the named concepts; record the section studied and one additional example.

Five-question lesson quiz

Answer independently. Core questions 1-3: at least 5/6. All five: at least 8/10, plus practical evidence.

1. How far does 8 units/second travel in 0.125 seconds?
2. What is the interval for 120 frames/second?
3. Why record hardware and workload with a timing result?
4. A serial frame has 2 ms input work, 5 ms simulation and 11 ms drawing. Does that model fit a 60-frame/s interval?
5. Why can a finite-value check accept a calculation that is wrong by a factor of 1,000?

Evidence to retain: predictions, code or worksheet, actual output, changed-case reasoning, quiz answers and corrections.

Module 02: Values, decisions and repetition

Prerequisites: Module 01.

Integration project: A bounded five-turn resource game. The full brief and review are in the Lab Workbook.

Lesson 02.1: Types and expressions

Learning outcome: Choose integers, real-valued numbers, strings and booleans appropriately.

Choose a representation for the fact

An inventory count describes whole items, so an integer is a natural representation. A player name is text, so it is a string. A door-open flag is true or false, so it is a Boolean. A continuously varying position may use floating-point numbers, which represent a finite set of approximations rather than every possible real number. These are modeling decisions before they are syntax decisions.

The value `12` and the text `"12"` look similar when printed, but their operations differ. Adding integers calculates a numeric sum; adding strings concatenates text. A text prompt produces text even when the user types digits. Decide how the text should be interpreted before combining it with a numeric quantity.

Expressions combine values according to rules

Parentheses make intended grouping visible. Multiplication ordinarily takes precedence over addition, so `2 + 3 * 4` produces 14, while `(2 + 3) * 4` produces 20. Use parentheses to explain a design rule, not just to force an answer by trial and error. The expression `count * price` is meaningful only after both values have passed the required interpretation and validation.

In Python, `/` performs division that can produce a non-integer result, while `//` is floor division. For negative values, floor is not the same as truncating toward zero. Do not choose an operator merely because it printed a whole number in one positive example. Our resource-cost calculation needs multiplication of whole counts, so it does not require an arbitrary rounding rule.

Parse first, then validate meaning

Parsing asks whether a text representation can be converted to the selected type. Validation asks whether the parsed value is allowed by the game. Text `"-2"` can be parsed as an integer, but a negative item purchase is excluded by our contract. Text `"two"` fails the parse itself. Both should lead to a useful message at the interaction boundary, but for different reasons.

`int(text)` either returns an integer or raises an exception for text it cannot interpret. A `try` block identifies an operation whose failure is expected. The matching `except ValueError` branch handles that specific failure. Do not catch every exception and pretend the program succeeded: a coding error elsewhere deserves investigation, not silent conversion to a default price.

Keep the calculation separate from user interaction. The small demonstration below uses fixed strings so that every learner gets the same input. The practical function returns a number or raises `ValueError`; a later command-line adapter will decide how to ask again or display an error. Reusability begins with that separation.

Precision is not solved by rounding everything

Some decimal fractions are not exactly representable in binary floating-point. An equality such as `0.1 + 0.2 == 0.3` can therefore be false. This is not a reason to insert rounding after every operation. Rounding changes the calculation and can accumulate error; first choose the representation and the acceptable comparison criterion.

For exact whole inventory counts and whole game-credit prices, use integers. If a design later introduces currency subdivisions, define the smallest unit explicitly rather than assuming a binary approximation is appropriate. For continuous movement, use a meaningful tolerance when comparing computed results. The mathematical and native-language volumes will revisit representation limits more deeply.

Learn from rejected input

A useful input error tells the player what was expected: “enter a whole-number count” is better than “bad.” Test zero, a normal positive value, negative text, fractional text and nonnumeric text. Zero cost may be legitimate; decide rather than accidentally rejecting it with an overly broad truthiness check.

Worked example

Predict before running `examples/example_02_1.py`.

```
count_text = "3"
price_text = "4"
print("text addition:", count_text + price_text)
try:
    count = int(count_text)
    price = int(price_text)
    if count < 0 or price < 0:
        raise ValueError("count and price must be nonnegative")
    print("numeric cost:", count * price)
except ValueError as error:
    print("Input rejected:", error)
```

The first print deliberately operates on strings and produces concatenated text. The conversion then creates numeric values. Raising a `ValueError` is an explicit refusal to accept a parsed but invalid value; it is not a crash when an appropriate caller handles it. Change one input to `"many"`, then `"-1"`, and explain where the two failures occur.

Expected demonstration output

```
text addition: 34
numeric cost: 12
```

Practical work and pass evidence

Implement `total_cost(count_text, unit_cost_text)` in `labs/student/lab_02_1.py`. Use whole-number text accepted by Python's integer conversion; reject malformed text and negative values with `ValueError`. Return the product without printing. Test count zero, price zero, ordinary values, surrounding spaces, negative values and fractional text.

Run `tools/check_labs.py --lesson 02.1` with your selected Python launcher. Also create a small interaction script that asks for two strings, calls the function and catches only the expected input error. Save an input/output table. Passing the supplied checks is necessary for the function exercise, but you must explain parsing versus domain validation independently.

Changed-case challenge

Add a maximum allowed purchase count of 99 in a separate experiment. Test 98, 99 and 100 without changing the meaning of zero.

Companion source study

[S02] Python: An Informal Introduction; [S04] Python Built-in Types; [S05] Python: Floating-Point Arithmetic; [S08] Python: Errors and Exceptions. Read the sections relevant to the named concepts; record the section studied and one additional example.

Five-question lesson quiz

Answer independently. Core questions 1-3: at least 5/6. All five: at least 8/10, plus practical evidence.

1. Why are "12" and 12 different?
2. What does "ab" + "cd" produce?
3. Would an exact item count normally use floating point? Why?
4. Why is `"-2"` a different failure case from `"two"` in a nonnegative item calculator?
5. A programmer fixes text input by concatenating and then converting: `int("3" + "4")`. Does that calculate the sum or product of 3 and 4?

Evidence to retain: predictions, code or worksheet, actual output, changed-case reasoning, quiz answers and corrections.

Lesson 02.2: Conditions and boundary cases

Learning outcome: Write decisions with complete and non-overlapping rules.

Translate the rule before choosing branches

Our door rule is: a character may enter when it has a key, or when it has both level at least 5 and stamina at least 10. Write the logical structure in words first. The key route is independent of the level-and-stamina route. A key holder with low stamina therefore passes under this particular rule. A different design may choose another rule, but code must implement the one actually specified.

The corresponding expression is `has_key or (level >= 5 and stamina >= 10)`. Parentheses show the grouping directly. Writing `(has_key or level >= 5) and stamina >= 10` changes the rule: now a key holder must also meet the stamina threshold. One familiar positive example may not reveal the difference, so choose a counterexample such as key=true, level=1, stamina=0.

Include the boundary itself

“At least 5” means 5 is accepted. `level > 5` excludes exactly that boundary. Test the value just below, exactly at and just above each threshold. With key/no-key and three values around each numeric threshold, this lesson has 2 x 3 x 3 = 18 nearby combinations. This is a small enough domain to enumerate rather than sample casually.

A truth table gives you independently chosen expected values before you execute the function. Do not generate the expected column using the exact expression being tested; that can reproduce the same mistake twice. Explain each row from the written rule, then use the program to check your explanation.

Choose complete, non-overlapping behavior

An `if`/`elif`/`else` chain selects one branch. Separate `if` statements can select more than one. Suppose one statement says “if stamina >= 10, open” and another says “if level < 5, close.” A character meeting one condition but not the other may trigger conflicting actions. A single decision expression followed by one open/closed result is easier to reason about here.

That does not mean separate conditions are always wrong. Independent actions can be appropriate when a command both records a statistic and plays a sound. The question is whether the actions are alternatives or independent consequences. State the relationship explicitly.

Truthiness is not a substitute for a type contract

A Boolean expresses two states. Other Python objects also have truth values, but that does not mean every nonempty string is an acceptable Boolean input. In particular, the nonempty string `"false"` is truthy. A parser for a human-facing configuration must interpret allowed text forms deliberately.

Our function contract assumes a genuine Boolean key flag and nonnegative integer level and stamina values. Validation belongs at the input boundary in a larger application. This keeps the lesson focused on the logical rule. State the assumption; do not claim the function validates arbitrary untrusted objects when it does not.

Short-circuit evaluation and readable decisions

For `A or B`, Python need not evaluate B once A is truthy. For `A and B`, it need not evaluate B once A is falsey. With pure comparisons that simply saves work. With expressions that also modify state, it can make behavior depend on which parts are skipped. Keep important changes outside a permission expression in this early design.

A readable predicate answers a question: `can_open(...)`. The caller can use its result to choose a message or update a door. The predicate should not secretly spend stamina while deciding whether entry is allowed. That separation will matter when an interface checks permissions repeatedly before the actual action.

Worked example

Predict before running `examples/example_02_2.py`.

```
cases = [(False, 4, 10), (False, 5, 10), (True, 1, 0)]
for has_key, level, stamina in cases:
    allowed = has_key or (level >= 5 and stamina >= 10)
    print(has_key, level, stamina, "->", allowed)
```

The three rows target distinct aspects of the rule: just below the level boundary, exactly at both boundaries, and the key override. Predict each answer from the English rule first. The loop only organizes repeated checks; it does not change permission values between rows.

Expected demonstration output

```
False 4 10 -> False
False 5 10 -> True
True 1 0 -> True
```

Practical work and pass evidence

Implement `can_open` in `labs/student/lab_02_2.py`. Prepare the 18-row table using key=false/true, level=4/5/6 and stamina=9/10/11. Add the key=true, level=1, stamina=0 counterexample. Explain why an expression requiring stamina for every key holder would fail that case.

Run the lesson checks and write one caller that prints an open/closed message based on the returned Boolean. Do not deduct stamina in the predicate. Pass by defending the grouping, the inclusive comparisons and every table row, not just by observing a green test run.

Changed-case challenge

Specify an alternative door rule in which using a key costs stamina. Write the new rule separately; do not silently change the original assignment.

Companion source study

[S06] Python: More Control Flow Tools. Read the sections relevant to the named concepts; record the section studied and one additional example.

Five-question lesson quiz

Answer independently. Core questions 1-3: at least 5/6. All five: at least 8/10, plus practical evidence.

1. Does gold > price allow buying an item when gold equals price?
2. Evaluate False or (True and False).
3. Why test immediately below, at and above a threshold?
4. For key=true, level=0 and stamina=0, compare the assigned expression with `(has_key or level >= 5) and stamina >= 10`.
5. Why is a nonempty string such as `"false"` unsafe to use directly as the Boolean key flag?

Evidence to retain: predictions, code or worksheet, actual output, changed-case reasoning, quiz answers and corrections.

Lesson 02.3: Loops and invariants

Learning outcome: Use repetition that terminates and preserves a stated property.

A loop repeats a rule, not an intention

A loop applies its body according to a defined control condition. A `for` loop over `range(10)` runs with values 0 through 9: ten iterations, not eleven. A `while` loop repeats while its condition remains true. If nothing changes the condition and nothing exits, it may never terminate. “Run until finished” requires a precise account of what makes progress toward finished.

For our resource simulation, the number of turns is an explicit nonnegative integer. A bounded `for` loop is therefore appropriate. Each turn first collects three units and then spends five only when five units are available. The order is part of the rule. Spending before collecting would produce a different trace.

State an invariant you can defend

An invariant is a property that remains true at a stated point during every iteration. Here the resource count is nonnegative after each complete turn. We can defend it in two cases: if there are fewer than five units after collection, nothing is spent; otherwise subtracting five leaves at least zero. Starting at zero establishes the property before the first turn.

For this exact collect-three/spend-five rule, the count after a complete turn is also less than five. Assume the previous count is 0 through 4; after adding three it is 3 through 7. If it is at least five, one subtraction leaves 0 through 2; otherwise it remains 3 or 4. This stronger property depends on our specific numbers and must not be reused unchanged for an arbitrary collection amount.

Distinguish state count from update count

A trace including the initial state has one more entry than the number of turns. Ten updates therefore produce eleven state entries. Zero updates still produce the initial state. This convention makes before/after comparisons straightforward, but it must be stated so that a reader does not mistake the eleven entries for eleven turns.

A sequence of snapshots and a sequence of events are different data. Later, a replay file stores events, while a debugger may store both events and resulting states. Naming them correctly prevents off-by-one errors in playback and testing.

Progress and termination

For a loop of ten iterations, a useful progress measure is the number of iterations remaining; it decreases each time the loop body completes. For an input-validation loop, progress may instead depend on the user eventually entering an accepted value or choosing quit. That is not a guaranteed finite loop unless an explicit retry limit or exit rule exists.

The current assignment has no user-driven stopping ambiguity. Validate the turn count before entering the loop. Negative values should not silently act like zero merely because an empty range results. The contract should distinguish a valid no-work request from an invalid request.

Check the rule with traces and changed parameters

Record resources before collection, after collection, after any spending and after the turn. The final answer alone can hide a wrong ordering. If your implementation spends unconditionally, a case with only three available units exposes the negative result immediately.

Once the fixed rule works, experiment in a separate function with a different collection amount. Reconsider the stronger less-than-five invariant instead of blindly asserting it. This is a first experience with the difference between an implementation-specific property and a more general requirement such as never creating negative inventory.

Worked example

Predict before running `examples/example_02_3.py`.

```
resources = 0
print("initial:", resources)
for turn in range(1, 6):
    resources = resources + 3
    if resources >= 5:
        resources = resources - 5
    print("turn", turn, "resources", resources)
```

The trace repeats a five-turn pattern for the chosen arithmetic, but the implementation should follow the rule rather than hard-code the pattern. The upper range endpoint 6 is excluded, so displayed turn labels are 1 through 5. An `if`, not an unconditional subtraction, protects the nonnegative resource rule.

Expected demonstration output

```
initial: 0
turn 1 resources 3
turn 2 resources 1
turn 3 resources 4
turn 4 resources 2
turn 5 resources 0
```

Practical work and pass evidence

Implement `simulate_resources(turns)` in `labs/student/lab_02_3.py`. Return the initial zero plus the value after every complete turn. Reject negative, non-integer and Boolean turn counts. The strict Boolean exclusion is a schema choice: a yes/no flag is not an acceptable turn count even though Python relates bool and int.

Trace ten turns manually and compare all eleven values. Test zero turns. Explain the nonnegative invariant and show a changed example that breaks an unconditional-spending version. Run the supplied checks, then create at least two additional tests with turn counts not used in the demonstration.

Changed-case challenge

Change collection to seven units per turn in a separate experiment. Explain why a single optional subtraction no longer guarantees a final count below five.

Companion source study

[S06] Python: More Control Flow Tools; [S04] Python Built-in Types. Read the sections relevant to the named concepts; record the section studied and one additional example.

Five-question lesson quiz

Answer independently. Core questions 1-3: at least 5/6. All five: at least 8/10, plus practical evidence.

1. How many values does range(2, 7) produce?
2. What must change in while remaining > 0 to make progress?
3. Give an invariant for an inventory limited to ten items.
4. How many entries should the returned trace contain for zero turns and for ten turns under this lesson's convention?
5. Does “resources never become negative” prove that exactly ten turns were simulated?

Evidence to retain: predictions, code or worksheet, actual output, changed-case reasoning, quiz answers and corrections.

Lesson 02.4: Functions and contracts

Learning outcome: Separate a calculation from interaction and state its valid inputs.

Give one operation an explicit interface

A function names an operation and receives its required inputs as parameters. Calling it supplies arguments for one execution. Our movement function takes position, velocity and elapsed time, then returns a new position. It does not decide how to obtain those inputs or how to display the answer. That makes the same operation usable from a terminal, a test or a future game loop.

A function definition does not perform the calculation immediately. Its body runs when called. Names created inside the call are local to that execution unless a different mechanism is deliberately used. Prefer explicit inputs and returned outputs over hidden global variables in this first design: a test can then describe a complete case without reconstructing unrelated application history.

Write the contract before the implementation

A contract identifies preconditions, the result and permitted side effects. For `advance(position, velocity, dt)`, the inputs must be finite numeric values, dt must be nonnegative and the computed output must remain finite. Negative velocity is allowed. The result is `position + velocity * dt`. The function performs no printing, prompting or mutation of external state.

These choices are part of this lesson's model, not universal laws about all movement systems. A more advanced integrator could deliberately support backward integration; this one does not. A bounded world could reject positions outside a map; this function has no map and therefore makes no such promise.

Return a result instead of showing it

`return` hands a value to the caller and ends that function execution. `print` writes a representation to an output stream. Printing a result does not make it the return value. A function that falls through without an explicit returned result returns `None`, which may cause a later arithmetic operation to fail.

The caller can store a return value, compare it or print it. This separation lets tests check numeric results without parsing terminal text. It also keeps a renderer from having to capture output just to find out where a character moved.

Validate assumptions, not just common inputs

A zero interval should return the original position. A stationary object with zero velocity should remain in place. Negative velocity should move in the opposite direction. Negative time is invalid. Infinity and not-a-number are also invalid for this contract. Even finite input values can produce a non-finite result through overflow, so the reference solution validates the result as well.

The introductory function accepts numeric inputs rather than arbitrary text. The input parser from Lesson 02.1 is a separate boundary. Do not insert a conversion such as `float(anything)` deep inside every function without considering what representations the interface is supposed to accept.

Errors should preserve the caller's choice

On invalid input, raise `ValueError` with a useful explanation. Do not silently clamp negative dt to zero because that hides a caller defect and changes the meaning of the request. A caller may catch the exception, display a message and keep the old state. Assignment happens only after the call returns successfully, which makes this pattern easy to reason about.

A type hint can document intent, but it does not automatically enforce runtime validation. A comment saying “dt is always valid” is not evidence either. Decide which layer checks the precondition and test that boundary.

Revisit the first module with functions

You can now turn the early paper operations into reusable functions: a movement trace, a swap, a displacement calculation and a frame-interval calculation. Their original reasoning remains the same. The wrappers simply make the operations independently callable and testable. Return to the optional starters from 01.1, 01.3 and 01.4 after completing this lesson.

Worked example

Predict before running `examples/example_02_4.py`.

```
import math

def advance(position, velocity, dt):
    if not math.isfinite(position) or not math.isfinite(velocity):
        raise ValueError("position and velocity must be finite")
    if not math.isfinite(dt) or dt < 0:
        raise ValueError("dt must be finite and nonnegative")
    result = position + velocity * dt
    if not math.isfinite(result):
        raise ValueError("new position overflow")
    return result

next_position = advance(2.0, -6.0, 0.25)
print("next position:", next_position)
```

The function does not print; the caller does. A negative velocity is permitted and yields a smaller position. Each validation branch rejects a different unsupported condition before a successful return. Explain what value `next_position` would keep in a caller that already had a value and then caught a failure from a new call.

Expected demonstration output

```
next position: 0.5
```

Practical work and pass evidence

Implement the documented function in `labs/student/lab_02_4.py`. Test normal movement, zero velocity, negative velocity, zero dt, negative dt, infinity, not-a-number and an overflowing result. Capture or inspect output to confirm the function itself does not print.

Write a contract comment naming units and side effects. Then revisit the optional first-module function starters and run their checks. Pass by explaining the difference between returning and printing, and by showing that a caller can retain its old value after catching an invalid update request.

Changed-case challenge

Write a second function that converts milliseconds to seconds and calls advance. Keep unit conversion at this boundary and reuse the original movement function unchanged.

Companion source study

[S06] Python: More Control Flow Tools; [S08] Python: Errors and Exceptions; [S14] Python math library. Read the sections relevant to the named concepts; record the section studied and one additional example.

Five-question lesson quiz

Answer independently. Core questions 1-3: at least 5/6. All five: at least 8/10, plus practical evidence.

1. What is the result of advance(2, 8, 0.25)?
2. Why is returning a value more reusable than only printing it?
3. What belongs in a function contract besides parameter names?
4. A function prints 12 but has no explicit return. What does `answer = function()` normally bind to answer?
5. Why check the result for finiteness when all three inputs are finite?

Evidence to retain: predictions, code or worksheet, actual output, changed-case reasoning, quiz answers and corrections.

Module 03: Organizing data and small programs

Prerequisites: Module 02.

Integration project: A data-driven simulator with validated persistence. The full brief and review are in the Lab Workbook.

Lesson 03.1: Sequences, dictionaries and identity

Learning outcome: Choose collections according to the operations the program needs.

Organize data around the questions you ask

A list preserves an ordered sequence of elements. A dictionary maps keys to values. Choose according to the operations your game needs, not according to which syntax looks shorter. A turn order may naturally be a list; finding a unit by its stable identifier may naturally be a dictionary. One game can use both representations for different purposes.

A list index identifies a position in the current ordering, not necessarily a persistent game entity. Removing or sorting entries can change which unit occupies index 2. A unit name is also a poor persistent identity if the player can rename the unit or two units can share a name. Give each unit a stable ID and treat the displayed name as ordinary editable data.

Distinguish rebinding from mutation

With `other = units`, both names refer to the same list. Appending through one name changes the object observed through the other. This is different from rebinding an integer name in Lesson 01.3. With `other = list(units)`, the outer list is new, but its elements still refer to the same objects. If those elements are dictionaries, mutating one dictionary is visible from both lists.

For the flat records in this lesson, copying each record with `dict(unit)` is sufficient to make independent copies of its ID, name and health fields, because the field values are immutable integers and strings. It would not recursively copy a nested mutable inventory list. That limitation is part of the contract; do not call a shallow copy a universal isolation mechanism.

Filter without corrupting the source

To select living units, build a new result containing units whose health is greater than zero. Avoid deleting entries from the same list you are currently walking through: the remaining indices can shift and cause an element to be skipped. A separate result is easier to reason about and preserves the original collection for comparison.

There are two legitimate filter designs: a new list of references to the original records, or a new list of copied records. This package's lab chooses copied flat records so that renaming a filtered record does not edit the source. Another application might deliberately choose shared identity. State the choice explicitly rather than discovering it accidentally through a bug.

Reject ambiguous identity

When constructing the ID index, reject duplicates. Silently replacing an earlier record with the later one may hide a data-entry or loading problem. The lookup can only be meaningful if the ID identifies one unit under the chosen schema. An empty collection is valid and should produce an empty index and an empty living-unit list.

An index contains a view of data at a point in time. If it contains copies, later edits to the source records will not automatically update those copies. A production design must decide who owns the authoritative record and whether other structures hold references, copies or IDs. This small lab teaches the tradeoff without pretending there is one representation suitable for every engine.

Test relationships as well as values

A test that only compares resulting text can miss aliasing. Record the source, create the filtered result, edit a result record and check whether the source changed. Likewise, verify that changing a name does not change the ID used for lookup. Check two living units with the same name but distinct IDs.

Keep the data set small enough to inspect manually: five units with a mixture of positive and zero health. The important work is not the volume of data; it is the clarity of identity, ownership and mutation rules. Those same questions later determine whether entity handles and resource references are reliable.

Worked example

Predict before running `examples/example_03_1.py`.

```
units = [
    {"id": 101, "name": "Scout", "health": 8},
    {"id": 102, "name": "Scout", "health": 0},
]
alias = units
copies = [dict(unit) for unit in units]
alias[0]["name"] = "Runner"
copies[0]["name"] = "Independent"
print("original name:", units[0]["name"])
print("copied name:", copies[0]["name"])
print("living IDs:", [unit["id"] for unit in units if unit["health"] > 0])
```

The alias shares the original list, so changing its first dictionary changes the source. The copied records are separate dictionaries, so their string-valued name edits remain separate. Two records start with the same name, but their IDs remain distinct. This example does not contain nested mutable fields; extending it to those fields requires another ownership decision.

Expected demonstration output

```
original name: Runner
copied name: Independent
living IDs: [101]
```

Practical work and pass evidence

Create five flat unit records and implement `index_units` and `living_units` in `labs/student/lab_03_1.py`. Use stable IDs, reject duplicate IDs and return independent shallow copies of flat records. Preserve the input collection while filtering.

Test an empty collection, all-dead units, duplicate names with different IDs and duplicate IDs. Rename a copied living unit and demonstrate that the original record is unchanged. Explain why the same technique would not isolate a nested inventory list. Keep the small input data and the before/after snapshots in your journal.

Changed-case challenge

Add a nested list of item IDs in a separate experiment and demonstrate the limit of a shallow record copy. Document one design that would make ownership clear.

Companion source study

[S07] Python: Data Structures; [S09] Python: Classes. Read the sections relevant to the named concepts; record the section studied and one additional example.

Five-question lesson quiz

Answer independently. Core questions 1-3: at least 5/6. All five: at least 8/10, plus practical evidence.

1. When is a dictionary preferable to a list?
2. Does b = a copy a Python list?
3. Why can a shallow copy still share changes?
4. After `b = list(a)`, where a contains dictionaries, does changing `b[0]["health"]` necessarily leave a unchanged?
5. Two units share the name Scout but have IDs 101 and 102. Must an ID index reject them?

Evidence to retain: predictions, code or worksheet, actual output, changed-case reasoning, quiz answers and corrections.

Lesson 03.2: Decomposition, modules and interfaces

Learning outcome: Divide a small game into parts that can be tested independently.

Separate policy from interaction

A resource rule answers a question such as “what balance remains after this spending request?” A terminal interface answers different questions: how to ask for input, how to display errors and when to stop. Combining all of these in one loop makes the arithmetic difficult to test without simulated typing. Separating them gives each part a narrower responsibility.

Our rule function receives two nonnegative integers. When the request is affordable it returns the remainder; otherwise it returns the unchanged balance. The interface may print “not enough resources,” but the rule function does not need to know what a terminal is. This is a small example of a boundary that later separates game simulation from rendering and platform services.

A module is an importable unit of code

A Python source file can be imported as a module. Other code then accesses its functions and constants through the module namespace or explicit imported names. Importing executes top-level statements, so a top-level call to `input()` can unexpectedly start a game during a test. Keep interaction behind a callable entry point and the `if __name__ == "__main__":` guard when a file serves both as a script and an importable module.

The guard asks whether the file is being run as the main entry point rather than imported in the usual way. It does not mean the whole module is inert: function definitions, imports and other top-level initialization still execute. Keep that initialization small, predictable and free of prompts or unrelated writes.

Specify the direction of dependencies

Let the interface import the simulation, not the simulation import the interface. Tests should be able to import the simulation without pulling in terminal prompts. If two modules import one another because both want to control the game loop, the design may lack a clear owner for orchestration. Move that coordination to an entry point that calls the lower-level operations.

A tiny application does not need a large framework to establish this direction. Two files can be enough: `simulation.py` with the rule and `main.py` with interaction. Add another module when it has a coherent responsibility, not just to make the project look professional.

Refactor behavior, not just text

Refactoring changes structure while preserving a defined behavior. Before moving code, save representative inputs and expected results. After the change, rerun those cases against the extracted function. When a refactor also changes a rule, separate and name that change so a reviewer can reason about it.

The first checkpoint is that importing the simulation neither prints nor prompts. The second is that the same input sequence produces the same state sequence. The third is that the command-line entry point still handles invalid input usefully. A clean folder structure alone proves none of these things.

Define failures at the appropriate layer

The rule rejects negative or non-integer values with `ValueError`; the parser handles text conversion. Under the strict lab schema, Boolean values are rejected as counts even though they participate in Python's integer hierarchy. Unaffordable but otherwise valid requests are not parser errors; they are valid game requests whose defined outcome is no change.

This distinction prevents a common design mistake: using exceptions for every ordinary game outcome while silently accepting truly invalid state. Decide what is an expected branch in the game and what violates an interface contract. Tests should make that distinction visible.

Worked example

Predict before running `examples/example_03_2.py`.

```
def spend(resources, cost):
    if resources < 0 or cost < 0:
        raise ValueError("nonnegative amounts required")
    if cost > resources:
        return resources
    return resources - cost

def main():
    resources = 8
    for cost in (5, 5, 0):
        resources = spend(resources, cost)
        print("cost:", cost, "remaining:", resources)

if __name__ == "__main__":
    main()
```

This miniature example uses trusted integer inputs; the student lab additionally validates the strict types. The three requests demonstrate affordable spending, a rejected purchase and a zero-cost request. Importing this example defines functions but does not call `main`. Running it directly calls the entry point once.

Expected demonstration output

```
cost: 5 remaining: 3
cost: 5 remaining: 3
cost: 0 remaining: 3
```

Practical work and pass evidence

Implement the strict `spend` contract in `labs/student/lab_03_2.py`. Refactor your earlier resource simulator into a rule module and a separate command-line entry point in `workspace/module03_simulator/`. The rule module must be importable without output or prompts.

Save a before-refactor trace and compare it with the after-refactor trace. Add direct function tests for exact affordability, insufficient balance, zero cost and invalid input. Pass by drawing the dependency direction and showing an independent test that uses the rule without launching the interface.

Changed-case challenge

Create a second caller that feeds a fixed list of requests without user input. It should reuse the same rule module rather than copy its arithmetic.

Companion source study

[S10] Python: Modules; [S08] Python: Errors and Exceptions. Read the sections relevant to the named concepts; record the section studied and one additional example.

Five-question lesson quiz

Answer independently. Core questions 1-3: at least 5/6. All five: at least 8/10, plus practical evidence.

1. What is an interface?
2. Why avoid starting a game as a side effect of importing simulation.py?
3. How does command-as-data design help replays?
4. Why can placing `input()` at module top level make automated tests hang?
5. A refactor changes a rejected purchase from no change to a negative balance. Is that merely an internal restructuring?

Evidence to retain: predictions, code or worksheet, actual output, changed-case reasoning, quiz answers and corrections.

Lesson 03.3: Files, serialization and validation

Learning outcome: Save structured data without assuming every file is trustworthy.

Serialization is a boundary crossing

Serialization converts structured information into a representation suitable for storage or transmission. JSON is a text format with numbers, strings, Booleans, null, arrays and objects. Parsing JSON establishes that the text has a recognizable structure; it does not establish that it describes a valid game state.

Our lesson save has exactly three fields: version, x and health. Version is integer 1; x is an integer from -1000 through 1000; health is an integer from 0 through 100. Additional fields are rejected in this small schema. A different format could allow extensions, but it must say how unknown fields are handled. The exact-key policy here makes mismatched versions easier to expose.

Validate shape, types, ranges and relationships

Validation proceeds in layers. First, the root must be an object/dictionary. Second, the required keys must be present and unexpected keys absent. Third, values must have the required types. Fourth, ranges and version must be supported. A numeric-looking string is not an integer under this schema. A Boolean is also not an acceptable health count or version, so the reference uses an exact integer type check rather than accepting the whole bool/int relationship.

Some states require relational checks too. The capstone later requires remaining energy plus spent turns to equal the starting energy. Checking each field individually would not detect every inconsistent pair. State validity is more than a list of independent minimums and maximums.

Build a candidate before replacing live state

Parse and validate into a new candidate. Replace the live state only after the complete operation succeeds. In `live = decode_save(text)`, assignment happens after the function returns. If decoding raises a handled error, the old binding can remain available. Do not clear the live state first and hope decoding succeeds.

Loading should not rewrite the save file. A truncated or malformed input must not destroy the known-good save you were trying to read. Test the failed-load path explicitly: save a valid state, attempt to load invalid data and verify that both the existing working state and the good saved bytes remain unchanged where the test applies.

Parsing limits and duplicate keys

Even text with valid JSON syntax can be unreasonable to process. This lesson limits input to 10,000 characters. The reference rejects duplicate object keys rather than silently choosing one occurrence. Its intended setting is small educational save data, not an unrestricted hostile-input processing service. Resource limits and parser behavior need deeper review in a production security boundary.

For trusted course fixtures, the lesson's checks are sufficient to expose common mistakes: missing fields, wrong types, unsupported versions, invalid ranges and truncated files. Never use `eval` to “parse” a save. The intended input is data, not instructions to execute. The JSON format does not require evaluating arbitrary Python expressions.

Writing safely is a separate problem

The lesson functions only encode and decode strings. A file wrapper controls where bytes are read or written. The capstone reference demonstrates writing a validated payload to a temporary file in the destination directory and replacing the destination only after the temporary write succeeds. This protects the old save from ordinary failed-write paths tested here; it is not a universal power-loss durability guarantee or a concurrent-writer transaction system.

Keep those claims distinct. A round trip proves that one valid state can be encoded and decoded by your implementation. It does not prove malformed data is rejected, interrupted writes are safe, all future versions are supported or the state matches the original game rules.

Worked example

Predict before running `examples/example_03_3.py`.

```
import json

original = {"version": 1, "x": 2, "health": 90}
text = json.dumps(original, sort_keys=True)
candidate = json.loads(text)
print("encoded:", text)
print("same values:", candidate == original)
print("independent dictionary:", candidate is not original)

live = dict(original)
try:
    live = json.loads('{"version":')
except json.JSONDecodeError:
    print("Rejected truncated JSON; live health:", live["health"])
```

The example demonstrates representation and assignment timing, not complete schema validation. `json.loads` can return many kinds of JSON values, so the lab must add explicit schema checks after parsing. The malformed input raises before the assignment to `live`, leaving the earlier dictionary available to the caller.

Expected demonstration output

```
encoded: {"health": 90, "version": 1, "x": 2}
same values: True
independent dictionary: True
Rejected truncated JSON; live health: 90
```

Practical work and pass evidence

Implement `encode_save` and `decode_save` in `labs/student/lab_03_3.py`, including the exact schema described above, duplicate-key rejection and the input-length limit. Return new dictionaries and do not write files inside these functions. Use a separate disposable workspace file to exercise a file wrapper.

Test a valid round trip, missing and extra fields, wrong types, a Boolean version, unsupported version, out-of-range health, duplicate keys, excessive text and truncated input. Keep the known-good state and save file unchanged after a failed load. Pass only after explaining why successful JSON parsing is not the same as valid game data.

Changed-case challenge

Propose a version-2 save with a new field. Write the policy for rejecting or migrating version 1 before implementing any migration.

Companion source study

[S11] Python json library; [S08] Python: Errors and Exceptions; [S15] Python os library: replace. Read the sections relevant to the named concepts; record the section studied and one additional example.

Five-question lesson quiz

Answer independently. Core questions 1-3: at least 5/6. All five: at least 8/10, plus practical evidence.

1. Does valid JSON imply a valid game state?
2. What does a round-trip test check?
3. Why include a format version?
4. Why is `{"version":1,"x":2,"health":-50}` invalid even though it is valid JSON?
5. What is the danger of resetting the live world to an empty dictionary before attempting a load?

Evidence to retain: predictions, code or worksheet, actual output, changed-case reasoning, quiz answers and corrections.

Lesson 03.4: Classes, composition and state machines

Learning outcome: Represent behavior without creating an unnecessary inheritance hierarchy.

Objects group related state and behavior

A class defines how a kind of object is initialized and which operations it exposes. An instance is one particular object created from that definition. A controller object might own a movement state and expose a method for processing events. This grouping can make ownership clear, but a class is not automatically a better solution than a small function and dictionary.

Start with the behavior you need. Our movement controller has three states: idle, moving and stunned. It receives events such as move, stop, hit and recover. Its value is not an inheritance hierarchy; it is a precise transition policy that prevents contradictory combinations such as simultaneously being idle and moving.

Write the transition table first

The five allowed state changes are idle+move to moving, moving+stop to idle, idle+hit to stunned, moving+hit to stunned, and stunned+recover to idle. Other events in a valid state leave it unchanged. An unknown current state is an error. This difference is deliberate: an unhandled player request can be a valid no-op, while an impossible stored state suggests corrupt data or a programming defect.

A table makes gaps visible. Does stop do anything while stunned? In this policy it does not. Does recover while moving reset movement? It does not. You may choose another policy for a different game, but record it before writing scattered conditionals. A state machine is useful because it names what is allowed, not because its code has a particular shape.

A transition can be a pure function

`transition(state, event)` can return the next state without changing any object. A controller method can then assign that result to its own state. This separates rule testing from storage. Tests can enumerate the table without needing an entire player object, while the object can still provide a convenient application interface.

When invalid state is rejected, assignment should not install a partial result. Keep the same candidate-before-commit pattern used for loading. The principle is broader than files: validate an operation before replacing the authoritative state.

Composition separates independent concerns

A character can contain a movement controller and an inventory. Stunned movement does not necessarily imply that the inventory ceases to exist. Making inventory a component or owned object allows its rules to be tested without the movement controller. Inheriting from classes named `MovingCharacterWithInventory` and `StunnedCharacterWithInventory` would entangle independent choices.

Composition means assembling behavior from parts with defined interfaces. It does not mean putting every field into a different class. A simple game may use a character record containing a state value and a list. The important question is which part owns each fact and how another part is allowed to change it.

Keep instance state independent

If each controller should start idle, initialize that state for each instance. Be careful with shared mutable objects defined at class or function-default scope; one character's changes can become visible in another unexpectedly. Test two controllers and verify that moving one does not move the other. Later engine storage may deliberately share immutable data, but that is an explicit design, not accidental aliasing.

Test all five allowed transitions and at least three forbidden transitions. A test showing that idle can become moving does not establish that stunned movement is prohibited. Finish by tracing a longer event sequence and explaining each result from the same table, so the implementation is not merely matching isolated examples.

Worked example

Predict before running `examples/example_03_4.py`.

```
TRANSITIONS = {
    ("idle", "move"): "moving",
    ("moving", "stop"): "idle",
    ("idle", "hit"): "stunned",
    ("moving", "hit"): "stunned",
    ("stunned", "recover"): "idle",
}

class Controller:
    def __init__(self):
        self.state = "idle"

    def on_event(self, event):
        self.state = TRANSITIONS.get((self.state, event), self.state)

controller = Controller()
for event in ("move", "hit", "move", "recover"):
    controller.on_event(event)
    print(event, "->", controller.state)
```

`self` refers to the particular controller receiving the method call. Its state starts independently in `__init__`. The table lookup defaults to the current state for an unlisted event. The demonstration keeps state valid by construction; the lab function additionally rejects an unknown starting state at its interface.

Expected demonstration output

```
move -> moving
hit -> stunned
move -> stunned
recover -> idle
```

Practical work and pass evidence

Implement `transition` in `labs/student/lab_03_4.py` with the exact five-entry table and the stated no-op/error policy. Build a small controller wrapper in your workspace and add an inventory field by composition rather than a hierarchy of movement subclasses.

Test every allowed transition, at least three forbidden transitions and an unknown state. Create two controller instances and demonstrate independence. Trace move, hit, move, recover, move, stop from idle. Pass by explaining both the state table and which object or function owns each change.

Changed-case challenge

Add an independent inventory operation while the controller is stunned. Decide explicitly whether it is allowed; do not let inheritance structure make the game-design decision accidentally.

Companion source study

[S09] Python: Classes; [S07] Python: Data Structures. Read the sections relevant to the named concepts; record the section studied and one additional example.

Five-question lesson quiz

Answer independently. Core questions 1-3: at least 5/6. All five: at least 8/10, plus practical evidence.

1. What does encapsulation protect?
2. Why can independent booleans create contradictory states?
3. When is composition useful?
4. Under the assigned table, what happens to a stunned controller receiving move, followed by recover?
5. Why test two controller instances instead of only one?

Evidence to retain: predictions, code or worksheet, actual output, changed-case reasoning, quiz answers and corrections.

Module 04: Debugging, testing and collaboration

Prerequisites: Modules 01-03.

Integration project: A reviewed, tested local release. The full brief and review are in the Lab Workbook.

Lesson 04.1: Debugging as an experiment

Learning outcome: Find a defect by narrowing evidence rather than making random edits.

A bug report begins with a violated expectation

“The health system is broken” is not enough to reproduce a defect. A useful report states the starting state, input sequence, expected result, actual result and environment. For a single hit of damage 3 applied to health 10, the contract expects health 7. Observing health 4 suggests a different operation occurred; it does not yet prove where the extra subtraction happened.

Keep the reproduction small. One character and one hit are better than a complicated match when they show the same failure. Remove irrelevant input, rendering and timing while preserving the difference. A minimal reproduction is an experiment whose outcome distinguishes correct behavior from the defect.

Find the first divergence

Instrument values at a few boundaries: before the event, after the simulation update and before drawing. If health is already 4 after simulation, changing the display format cannot fix the first incorrect state. If simulation reports 7 but drawing later changes it to 4, inspect whether the presentation layer is applying damage a second time.

You can use print tracing or a debugger breakpoint. Both are ways to observe execution, not substitutes for a hypothesis. Choose a location because it can distinguish competing explanations. Do not add hundreds of prints and hope the volume of output reveals the cause by itself.

Form a falsifiable hypothesis

A useful hypothesis is “damage is subtracted in both the update handler and the render handler.” It predicts that drawing twice after one hit will reduce health again. Another hypothesis is “the queued hit appears twice,” which predicts two event records before drawing. These explanations need different repairs, even if both produce the same final health.

Change one relevant factor at a time and record the result. Randomly moving statements until the symptom disappears can introduce a new defect or hide the original one for the observed input. A repair is more convincing when the failing test first demonstrates the issue and then passes after the smallest justified change.

Regression tests preserve what you learned

A regression test encodes the behavior that failed so a future change can reveal its return. For the pure damage rule, assert that one event applied to health 10 with damage 3 returns 7. Also test lethal damage, zero damage and already-zero health. Invalid negative damage is not healing under this contract; it must be rejected.

A focused test is not proof that every event is dispatched once in the complete game. That broader risk needs an integration test showing that one command produces one update. Separate the narrow arithmetic contract from the wiring contract. A correct function called twice can still make a broken game.

Do not debug by suppressing all evidence

Catching every exception and continuing with an arbitrary value can make a program look stable while corrupting its state. Removing an assertion because it fails may silence the detector instead of repairing the violation. The appropriate question is whether the contract, implementation or test is wrong, and the answer requires independent reasoning from the specification.

Keep expected and actual results visible. Preserve the failing input with the fix. Explain why the modified code corrects the rule rather than merely matching one result. A reviewer should be able to introduce the old defect in a disposable copy and observe the regression test fail again.

Learn from the failure without inflating the claim

The included mutation demonstration intentionally doubles damage and verifies that one focused assertion rejects it. This shows the test can detect that known mutation. It does not establish that the test suite finds every possible defect. Be equally precise in your own lab record: name the seeded bug, the detecting test and the boundary of what was checked.

Worked example

Predict before running `examples/example_04_1.py`.

```
def buggy_damage(health, damage):
    health = max(0, health - damage)
    health = max(0, health - damage)  # Deliberate duplicate update.
    return health

starting_health = 10
one_hit = 3
expected_health = 7
actual_health = buggy_damage(starting_health, one_hit)
print("expected:", expected_health)
print("actual:", actual_health)
print("defect observed:", actual_health != expected_health)
```

This example is deliberately wrong and labeled as such. The second subtraction is another update, not display work. Trace the intermediate value after the first subtraction to identify the earliest statement that makes the result disagree with the one-event contract. Do not treat running this demonstration as completing the repair lab.

Expected demonstration output

```
expected: 7
actual: 4
defect observed: True
```

Practical work and pass evidence

Introduce a double-damage bug into a disposable copy of your simulator. Write a minimal report with health 10, damage 3, expected 7 and actual behavior. Trace or break at the first incorrect update. Add a failing test before repairing the copied simulator.

Implement `apply_damage` in `labs/student/lab_04_1.py`: nonnegative integer inputs, one subtraction and a zero floor. Demonstrate that the repaired version passes, and that temporarily restoring the known bug makes a relevant test fail. The supplied `tools/mutation_demo.py` illustrates this method; your own bug report and regression test remain required.

Changed-case challenge

Construct an integration bug where a correct damage function is called twice. Explain why fixing the arithmetic function would be the wrong repair.

Companion source study

[S08] Python: Errors and Exceptions; [S12] Python unittest library. Read the sections relevant to the named concepts; record the section studied and one additional example.

Five-question lesson quiz

Answer independently. Core questions 1-3: at least 5/6. All five: at least 8/10, plus practical evidence.

1. What information makes a bug report actionable?
2. Why inspect the earliest wrong state?
3. What is a regression test?
4. A correct damage function returns 7 for health 10 and damage 3, but the whole game reaches 4. What layer should you investigate next?
5. Why should a regression test fail before the fix is applied?

Evidence to retain: predictions, code or worksheet, actual output, changed-case reasoning, quiz answers and corrections.

Lesson 04.2: Unit, integration and property tests

Learning outcome: Choose tests that challenge the contract instead of repeating the implementation.

Test a contract, not a copy of the code

An automated test sets up a known case, invokes the operation and compares the outcome with an expectation. The expected result should come from a rule, derivation or independent reference. If the test calculates its expected value by copying the implementation, the same error can exist on both sides and remain invisible.

For a one-dimensional board of size 4, valid positions are 0 through 3. Moving right from 3 is rejected under our whole-move rejection policy, so the result remains 3. An expected result derived from the written coordinate rule is more independent than another copy of the same potentially incorrect comparison.

Unit, integration and property-style checks

A unit test targets a small operation, such as spending resources. An integration test checks cooperating parts, such as encoding a state, writing it, reading it and decoding it. A property-style check tests a relation across many inputs: every accepted movement result remains within the board, for example.

The simple property loops in this package enumerate bounded input sets using ordinary Python. They are not exhaustive proofs over all integers and they do not require a third-party property-testing library. Record the tested domain. A broad-looking loop can still omit the exact boundary that matters.

Properties need more than one dimension

An implementation that always returns the starting position stays in bounds, but fails to move when it should. An implementation that always returns zero also stays in bounds for many tests. Therefore combine safety properties with specific expected transitions and progress requirements. Correctness is not one convenient invariant; it is satisfaction of the relevant contract.

Similarly, a save round trip alone may allow an encoder and decoder to share an incorrect schema. Include independent known fixtures and rejection cases. Check that loading malformed data fails without modifying the previous valid state. Separate evidence for ordinary operation from evidence for failure behavior.

Make tests isolated and repeatable

Each test should establish the state it needs rather than depend on the previous test's side effects. Use fresh lists or dictionaries for mutable data. File tests should use a temporary directory or an explicitly disposable workspace rather than overwrite a personal file. Clean up only files the test created.

Use descriptive test names such as `test_right_edge_rejects_move` rather than `test_7`. When a failure occurs, the name should indicate the violated rule. Keep assertions small enough that the failure report identifies which property disagreed. A passing suite is useful evidence, but its value depends on the cases and the independence of their expectations.

Learn the runner without confusing it with your work

Python's `unittest` framework discovers and executes test methods. The supplied checker runs package-authored contract tests against student or reference implementations. Reference mode verifies the supplied answers; it does not check your implementation and cannot certify learning. Student mode points at the editable starter files, which intentionally fail until completed.

You must also write at least twelve tests spanning motion, resource spending and save/load. Do not submit the supplied tests unchanged as learner-authored work. Use them as examples of the interface, then challenge it with additional boundaries and failure conditions.

Test the tests with a known mutation

In a disposable copy, replace `< size` with `<= size`, or subtract damage twice. A relevant test should fail. If it does not, the suite has a gap for that mutation or is not executing the code you think it is. Restore the correct implementation after the experiment and record both outcomes. Do not leave intentional bugs in the version you later package.

Worked example

Predict before running `examples/example_04_2.py`.

```
def bounded_move(position, delta, size):
    candidate = position + delta
    if 0 <= candidate < size:
        return candidate
    return position

cases = [(2, 1, 4, 3), (3, 1, 4, 3), (0, -1, 4, 0)]
for position, delta, size, expected in cases:
    actual = bounded_move(position, delta, size)
    assert actual == expected
print("three independent boundary cases passed")

for position in range(4):
    for delta in (-1, 0, 1):
        assert 0 <= bounded_move(position, delta, 4) < 4
print("twelve bounded-input property cases passed")
```

The three explicit cases test a successful move and both edges. The property loop tests twelve additional input combinations, but some overlap the explicit cases; do not report them as fifteen unique input cases without checking. Each assertion uses ordinary Python assertion semantics; the fuller package checker uses unittest assertions and should be run without intentionally disabling checks.

Expected demonstration output

```
three independent boundary cases passed
twelve bounded-input property cases passed
```

Practical work and pass evidence

Implement `bounded_move` in `labs/student/lab_04_2.py` with the documented whole-move rejection policy. The starting size must be positive and the starting position inside the board. Then write at least twelve learner-owned tests in `workspace/test_my_rules.py` across motion, spending and save/load, including at least one bounded property loop.

Record each test's requirement and independently derived expected result. Introduce an off-by-one or double-update mutation in a disposable copy and show a named test detects it. Restore the correct code. A complete submission includes tests, a successful run and the mutation experiment, not only the supplied checker output.

Changed-case challenge

Write a deliberately useless implementation that never moves. Identify which property tests it passes and which expected-transition test rejects it.

Companion source study

[S12] Python unittest library. Read the sections relevant to the named concepts; record the section studied and one additional example.

Five-question lesson quiz

Answer independently. Core questions 1-3: at least 5/6. All five: at least 8/10, plus practical evidence.

1. How does an integration test differ from a unit test?
2. Why is copying the implementation into the test oracle weak?
3. What does a test suite passing establish?
4. Why is a bounds-only property insufficient for a movement function?
5. Does `--reference` on the supplied checker validate your edited student files?

Evidence to retain: predictions, code or worksheet, actual output, changed-case reasoning, quiz answers and corrections.

Lesson 04.3: Git, branches and recoverable changes

Learning outcome: Use small commits to preserve reasoning and collaborate safely.

History should preserve reasoning

Version control records meaningful states of a project and the changes connecting them. A commit is most useful when it represents a focused change with a message explaining its purpose. “Stuff” or “final final” does not tell a future reviewer why a rule changed. Commit after a coherent improvement with tests and evidence, not merely after a large amount of typing.

A local Git repository is enough for this lesson. A hosting account, public upload and remote push are not required. Keep personal credentials, private documents, real player data and generated temporary files out of the repository. Review the staged content before making a commit rather than assuming every file in the directory belongs in project history.

Working tree, index and commit

The working tree contains the files you are editing. The staging area, also called the index, selects the content for the next commit. The commit records that staged state. Editing a file after staging it does not automatically stage the later edit. Inspect both the unstaged difference and the staged difference when the distinction matters.

Use `git status` to see the current situation, `git diff` for unstaged changes and `git diff --staged` for staged changes. Use `git add` with explicit file paths while learning. A broad add command can include unrelated material that happened to be present. A `.gitignore` rule helps prevent accidental tracking of future generated files, but it does not erase already committed content.

Branches isolate a line of work

A branch gives a name to a line of development. Start from a known baseline, create a feature branch, make a focused change and test it. Merging incorporates the branch's work into the destination branch. A fast-forward merge is possible when the destination has not diverged from the branch's ancestor; it need not create an extra merge commit.

A branch is not a separate backup of every external file on your computer. Git tracks the repository's content and history under its rules. Keep ordinary backups where appropriate, especially before learning destructive operations. This lesson deliberately avoids force pushes and hard-reset cleanup as a normal recovery workflow.

Revert a mistake without rewriting the lesson

`git revert` creates a new commit that reverses a selected earlier change. The mistaken commit remains visible in history, along with the correction. This is useful for demonstrating why the change was wrong and how it was repaired. More complex histories can produce conflicts during a revert; the beginner exercise chooses a small nonconflicting history.

Do the exercise in a disposable new repository, not an employer repository or an important existing project. The supplied sandbox tool creates its own temporary directory and uses command-scoped identity, leaving global configuration alone. Its automated demonstration is a reference; you must still perform and explain your own three-commit and branch exercise.

A concrete safe sequence

In your new practice folder, initialize a repository with a main branch, add a small rules file and make a baseline commit. Create a `boundary-fix` branch, add an explicit zero-health rule, commit and merge it. Then make an intentionally wrong damage change and revert that new commit. Inspect the final file and log. The final rules should match the intended pre-mistake version while the history retains the mistake and revert.

Configure author identity only according to your own environment and privacy choices. Do not copy someone else's name or email into commits. Use local repository settings for a practice repository when needed, and inspect their scope. The companion worksheet supplies the shell commands and asks you to record what changed at each stage.

Worked example

Predict before running `examples/example_04_3.py`.

```
from pathlib import Path
import runpy

package_root = Path(__file__).resolve().parents[1]
print("Running the isolated Git demonstration.")
runpy.run_path(str(package_root / "tools" / "git_sandbox.py"),
               run_name="__main__")
```

This Python wrapper launches the supplied temporary-repository demonstration. It does not run commands against your current Git project. Commit IDs in its output vary because commit metadata and timing vary. The expected evidence is a successful branch/merge/revert sequence and restored rules, not a specific hash copied from a screenshot. Git must be installed for this example.

Expected demonstration output

```
Temporary repository only; no existing course files changed.
[commit graph; commit IDs vary]
Restored rules:
damage=3
health_min=0
PASS: branch, merge, revert and clean working tree.
```

Practical work and pass evidence

Use the Git worksheet in `templates/GIT_LAB.md` inside a new disposable practice directory. Make three focused commits, implement a change on a branch, merge it and revert an intentionally bad commit. Before committing, inspect staged differences. Save the log, final rules, commands and an explanation of the staging area in `workspace/04_3_git_evidence.md`.

Run the supplied sandbox demonstration as a comparison, not as a substitute for your own exercise. Pass by identifying the exact content of each commit, explaining why the revert adds history and showing that your working tree is clean after completion.

Changed-case challenge

Stage a file, edit it again, then compare git diff with git diff --staged. Explain which version would enter the next commit.

Companion source study

[S13] Pro Git: Recording Changes to the Repository; [S16] Git revert documentation. Read the sections relevant to the named concepts; record the section studied and one additional example.

Five-question lesson quiz

Answer independently. Core questions 1-3: at least 5/6. All five: at least 8/10, plus practical evidence.

1. What does the staging area control?
2. Why should secrets not be committed even if later deleted?
3. What makes a useful commit message?
4. You stage version A of a file, edit it to version B, and commit without staging again. Which content is committed?
5. Does reverting a mistaken commit normally remove that commit from the project history?

Evidence to retain: predictions, code or worksheet, actual output, changed-case reasoning, quiz answers and corrections.

Lesson 04.4: Review, documentation and responsible AI help

Learning outcome: Explain a change and verify assistance without surrendering understanding.

Review starts with the requirement

A reviewer cannot judge a change solely from whether the code looks tidy. Ask what behavior is required, which cases changed, which assumptions are trusted and what evidence supports the claim. A ten-line function can violate an important boundary even when its variable names are clear.

For a purchasing rule, determine whether exact affordability is allowed, what zero cost means and how negative values are handled. A review comment such as “use a shorter expression” is secondary if the comparison rejects a purchase costing exactly the available balance. Prioritize correctness and clear contracts before stylistic preferences.

Read a diff as a change in behavior

A diff shows what changed between versions. Trace the changed lines using at least one normal input, one boundary input and one invalid input. Ask whether the tests distinguish the old behavior from the intended new behavior. A test added with an expectation copied from the changed code is weak evidence if nobody checked it against the requirement.

Review side effects too. A function that previously only returned a result might now write a file or print a message. That may break callers even when the numeric output is unchanged. The interface includes effects and failure behavior, not only parameter names.

Write instructions another person can execute

A README should identify the required interpreter, the working directory, exact commands, expected success signal, controls, test command and known limitations. A person starting from a clean extraction should not need your memory of an unrecorded setup step. “Run the project” is not an executable instruction.

Test the README by following it in a fresh directory. Keep generated state and hidden editor configuration from becoming undeclared prerequisites. Record the actual interpreter used rather than claiming compatibility with every possible environment. When a tester encounters a failure, improve the instructions or implementation based on evidence.

Assistance must not replace understanding

AI, documentation and peers can help explain an error, suggest cases or review a design. Record material assistance and verify the result. Predict the operation before accepting generated code; explain each changed line; run tests and create a changed example without following the answer. A passing test suite does not establish that you understand an implementation well enough to maintain it.

Do not upload private employer source, credentials, proprietary SDK material or another person's personal data without permission. Use a minimal non-sensitive reproduction when asking for help. The programming exercise is an opportunity to learn better boundaries, not a reason to relax them.

Separate what the reviewer knows from what is assumed

A reviewer may verify a code path by reading it, run selected tests or reproduce the whole package. These are different evidence levels. State which happened. “Reviewed” should not imply that every supported platform was physically tested when only a short diff was inspected.

Likewise, acknowledge limits in your own handoff. You may have run the program on one machine and asked one person to try it. That is a useful start, not a comprehensive usability study. The purpose is to build habits of precise claims and repeatable evidence.

Practice a live change

After reviewing, close the reference and implement a related variation, such as changing a threshold from 5 to 6 or adding a zero-cost case. Predict which tests should change and which should not. If you cannot explain the effect without copying a solution, return to the underlying lesson. A review is successful when it improves both the artifact and your ability to reason about it.

Worked example

Predict before running `examples/example_04_4.py`.

```
def proposed_purchase(balance, cost):
    # Deliberate review candidate: inspect the boundary comparison.
    if balance > cost:
        return balance - cost
    return balance

for balance, cost in ((6, 5), (5, 5), (4, 5)):
    print("balance/cost/result:", balance, cost,
          proposed_purchase(balance, cost))
print("Review question: should exact affordability be accepted?")
```

The candidate is intentionally not endorsed as a correct purchasing rule. Its normal case works, which can distract from the exact-affordability boundary. Compare the result with the original nonnegative spending contract. Also note that the demonstration does not validate negative inputs; a complete review must identify that missing boundary separately.

Expected demonstration output

```
balance/cost/result: 6 5 1
balance/cost/result: 5 5 5
balance/cost/result: 4 5 4
Review question: should exact affordability be accepted?
```

Practical work and pass evidence

Review a roughly ten-line change using `templates/CODE_REVIEW.md`. Address requirement, correctness, boundaries, side effects, clarity and tests. Produce a README for your simulator that another person can follow from a clean folder. Record assistance and provenance in the template.

Perform one small related change without copying the reference, predict its effects and update the tests. This is an evidence-based lab; there is no automatic function checker for it. Pass by explaining every changed line and by recording a clean-start result from another person or a clearly labeled self-review when independent review is not yet available.

Changed-case challenge

Review a change that moves input() into a previously pure function. Explain the interface regression even when normal interactive play still appears to work.

Companion source study

[S01] Harvard CS50 Introduction to Programming with Python; [S12] Python unittest library; [S13] Pro Git: Recording Changes to the Repository. Read the sections relevant to the named concepts; record the section studied and one additional example.

Five-question lesson quiz

Answer independently. Core questions 1-3: at least 5/6. All five: at least 8/10, plus practical evidence.

1. What should a review criticize?
2. Why is compiling AI-generated code insufficient validation?
3. What demonstrates understanding beyond copying a solution?
4. What is the first requirement to clarify when reviewing the purchase candidate with balance=5 and cost=5?
5. Why is “the AI wrote it and all supplied tests pass” insufficient for this course's review check?

Evidence to retain: predictions, code or worksheet, actual output, changed-case reasoning, quiz answers and corrections.

Module 05: A complete first game

Prerequisites: Modules 01-04.

Integration project: Training Yard: the Gate A first-game release. The full brief and review are in the Lab Workbook.

Lesson 05.1: The update loop and command model

Learning outcome: Separate simulation ticks from input collection and presentation.

One command, one simulation decision

A turn-based game loop can be described as read a command, decide its effect, produce the next state and display that state. The display may also be refreshed independently, but it must not apply another movement. We choose a turn-based terminal interface so that sequencing is visible without introducing real-time scheduling or graphics APIs prematurely.

The update function should be callable without a terminal. A test can supply `d` and inspect the returned position. A replay can feed a sequence of commands. The interactive adapter can call the same function after reading input. This is a small engine-like boundary built from the pure functions you already know.

Define a coordinate convention

Our map has ten columns and ten rows. Coordinates range from 0 through 9. The origin is the upper-left cell. x increases rightward and y increases downward. Thus w requests `(0,-1)`, a requests `(-1,0)`, s requests `(0,1)` and d requests `(1,0)`. These are chosen screen-grid conventions, not a universal definition of every mathematical or 3D coordinate system.

Write the convention beside the map. A movement error may be a sign mismatch rather than a broken loop. Test one step in each direction from an interior position so that an edge does not hide the intended change.

Normalize at the input boundary

A player may type `D` or add spaces. The normalization function strips surrounding whitespace and converts the command to lowercase. It does not execute a game action. The rule then looks up the normalized command in a small movement table.

Unknown commands produce no movement in this helper model. The interactive adapter should explain the supported controls. Do not interpret arbitrary text as code or silently substitute a different command. The capstone also limits command length so accidental oversized input is rejected explicitly.

Generate a candidate, then accept it

Calculate the candidate position before assigning it as the new state. If either coordinate is outside the map, reject the entire movement. Do not update x first and discover afterward that y is invalid. The whole-move rule makes failure predictable and prevents partial changes from leaking through rejection paths.

For the pure helper, returning the original tuple is a straightforward no-op. In a larger state object, other fields such as energy must also remain unchanged when the rule says a rejected move costs nothing. We will add that consistency requirement during integration rather than bury it inside drawing.

Render by observation

A renderer walks the rows and columns and chooses a symbol for each cell. P marks the player, E marks the exit and a dot marks open space. When the player stands on the exit, give P display priority so there is still one visible player marker. The underlying game state, not the displayed symbol, determines victory.

Rendering returns a string. The terminal adapter prints it. A test can compare the string with a known small map and verify that repeated rendering does not change position. A 3x3 example is easier to audit than a 10x10 screenshot, while the actual lab still builds the specified 10x10 board.

Assemble the smallest complete loop first

Begin with movement and display, then add input normalization and useful unknown-command feedback. Do not add physics, networking or animated graphics to avoid finishing a simple loop. Keep a written list of what the current stage does not yet implement: obstacles, energy and terminal win/loss policy arrive next. A clear limitation is better than a half-implemented feature claiming to be complete.

Worked example

Predict before running `examples/example_05_1.py`.

```
position = (1, 1)
command = " D ".strip().lower()
deltas = {"w": (0, -1), "a": (-1, 0), "s": (0, 1), "d": (1, 0)}
dx, dy = deltas[command]
candidate = (position[0] + dx, position[1] + dy)
if 0 <= candidate[0] < 3 and 0 <= candidate[1] < 3:
    position = candidate
for y in range(3):
    row = ""
    for x in range(3):
        row += "P" if (x, y) == position else (
            "E" if (x, y) == (2, 2) else ".")
    print(row)
print("position after drawing:", position)
```

The example uses a 3x3 board for a compact trace. It performs exactly one candidate update, then only reads the result while producing rows. Drawing the rows again would not change the tuple. The exit is a display marker here; reaching it becomes a state transition in the next lesson.

Expected demonstration output

```
...
..P
..E
position after drawing: (2, 1)
```

Practical work and pass evidence

Implement `normalise_command`, `step_position` and `render_grid` in `labs/student/lab_05_1.py`. The default board is 10x10; the helpers also accept a small square size for tests. Draw P at the player and E at the bottom-right exit unless the player occupies it. Unknown or out-of-bounds movement returns the unchanged valid position.

Connect these helpers in a terminal adapter in your workspace. Test one movement per input and repeated drawing with no updates. Compare a known 3x3 rendering, then run the actual 10x10 version. Pass by showing that the update can be called without input or printing and that the display is read-only.

Changed-case challenge

Add a second read-only display function that reports numeric coordinates. Calling both displays must still produce only one update per command.

Companion source study

[S06] Python: More Control Flow Tools; [S07] Python: Data Structures; [S10] Python: Modules. Read the sections relevant to the named concepts; record the section studied and one additional example.

Five-question lesson quiz

Answer independently. Core questions 1-3: at least 5/6. All five: at least 8/10, plus practical evidence.

1. Are a simulation tick and a rendered frame necessarily identical?
2. What should happen when render is called twice without an update?
3. Why is headless simulation useful?
4. With an upper-left origin and y increasing downward, which coordinate change represents w/up?
5. Why should victory depend on the state rather than whether an E symbol remains visible after drawing?

Evidence to retain: predictions, code or worksheet, actual output, changed-case reasoning, quiz answers and corrections.

Lesson 05.2: Collision rules and complete behavior

Learning outcome: Specify edge cases before implementing a playable rule.

Turn a partial demonstration into a specified game

Movement on an empty board is not yet a complete game. Add an explicit goal, obstacles and rules for what happens after victory. The lesson model uses five fixed obstacles: (1,1), (3,2), (4,4), (7,6) and (8,8). The player starts at (0,0) and the goal is (9,9). A clear route exists along the top edge and then down the right edge.

Do not rely on the player to avoid unsupported situations. Define behavior at each edge, against a wall, for unknown commands and after the round ends. “Nothing should happen” must specify whether position, energy, turn count, messages or replay history are allowed to change. The helper model contains only x, y and won; the capstone expands that state explicitly.

Reject the whole candidate

Construct a candidate coordinate pair and check it against both map bounds and obstacles before changing the authoritative world. A blocked movement leaves x, y and won unchanged. Return a new flat state dictionary so the original input remains available for comparison; do not mutate it and then try to undo selected fields.

This is a small transaction-like design pattern: prepare, validate, commit. We are not claiming database transaction guarantees; we are using the idea to make one in-memory operation easier to reason about. The pattern becomes more important as a movement also affects resources and status.

Terminal states need a policy

When an accepted move reaches (9,9), set won=true. In the lesson helper, all later commands leave that world unchanged. The capstone adds explicit restart and quit behavior around its richer state machine. Keep the difference documented instead of accidentally making the helper's terminal policy inconsistent with its tests.

A restart should create a fresh initial state. Reusing a mutated starting dictionary can carry old information into the new round. A terminal status should also not cause a program to spin endlessly printing the same screen without input or exit handling. Separate the world transition from the terminal adapter's decision to leave the application.

Add a loss condition for the capstone

The completed reference game begins with 25 energy. A legal move or wait spends one unit; rejected wall/edge moves and unknown commands spend none. Reaching the goal wins even when that last legal action spends the final unit. Otherwise energy zero causes a loss. After won/lost/quit, movement is a no-op; r creates a fresh round.

The order of goal and exhaustion checks matters. A state at the goal with zero energy is a win under the specified priority. Checking exhaustion first would produce a different game. Tests must include this tie case instead of assuming one outcome is universally correct.

Invariants connect fields

For this capstone, remaining energy equals 25 minus accepted-action turn count. Both values must be nonnegative and the turn count cannot exceed 25 within one round. The player must stay on an open map cell. A won state must be at the goal; a lost state has zero energy and is not at the goal. These are relational requirements, not only type checks.

Do not add energy to the simpler lesson helper until it passes its own contract. Then integrate the richer rules in your capstone project with new tests. Incremental construction gives you a known-good baseline and isolates whether a failure came from collision handling, resource accounting or terminal-state policy.

Worked example

Predict before running `examples/example_05_2.py`.

```
world = {"x": 1, "y": 0, "won": False}
obstacles = {(1, 1), (3, 2), (4, 4), (7, 6), (8, 8)}
candidate = (world["x"], world["y"] + 1)
next_world = dict(world)
if (0 <= candidate[0] < 10 and 0 <= candidate[1] < 10
        and candidate not in obstacles):
    next_world["x"], next_world["y"] = candidate
    next_world["won"] = candidate == (9, 9)
print("attempted cell:", candidate)
print("original:", world)
print("result:", next_world)
```

The attempted downward step hits the obstacle at (1,1). The candidate is rejected before any coordinate assignment, so the complete state remains unchanged. The new dictionary is independent at the flat-field level; the helper's fields are integers and a Boolean. This example does not yet contain the capstone energy fields.

Expected demonstration output

```
attempted cell: (1, 1)
original: {'x': 1, 'y': 0, 'won': False}
result: {'x': 1, 'y': 0, 'won': False}
```

Practical work and pass evidence

Implement `step_world` in `labs/student/lab_05_2.py` with the fixed map, pure flat-state return and terminal win policy. Test all four edges, all five obstacles, a normal move, reaching the goal and commands after victory. Preserve the input dictionary and reject the complete move rather than changing one coordinate early.

Once the helper passes, implement the capstone energy/loss/restart requirements from `CAPSTONE_SPEC.md` in your own game. Keep separate evidence for the helper and the richer game. Pass by explaining collision rejection, terminal priority and a live restart that leaves no stale round state.

Changed-case challenge

Reach the goal on exactly the last energy unit by waiting seven times, moving right nine times and down nine times. Predict the final status before testing.

Companion source study

[S06] Python: More Control Flow Tools; [S09] Python: Classes. Read the sections relevant to the named concepts; record the section studied and one additional example.

Five-question lesson quiz

Answer independently. Core questions 1-3: at least 5/6. All five: at least 8/10, plus practical evidence.

1. What is missing from "move right increases x"?
2. Under the reject policy, what happens at x = 9 on a 0-9 grid?
3. Why validate before applying a multi-part state change?
4. Why is changing x before checking whether the full candidate cell is blocked risky?
5. Under the capstone rule, what status results when the final energy unit is spent entering the goal?

Evidence to retain: predictions, code or worksheet, actual output, changed-case reasoning, quiz answers and corrections.

Lesson 05.3: Deterministic replay and simple measurement

Learning outcome: Reproduce a run and measure the work actually performed.

Determinism is a claim about specified inputs

For our pure lesson model, the same initial state, map, rules and ordered commands should produce the same final state. This statement includes more than a list of keyboard letters. Changing the map, starting position or rule version changes the input conditions even when the command text is identical.

The lesson helper starts from a fixed world, so its replay function can accept only the command sequence. The capstone replay file is more explicit: it records a format version, rule/map identifier, initial state and ordered commands. This makes the replay's assumptions inspectable instead of depending on undocumented defaults.

Replay tests more than the final screenshot

Record a twenty-command session and replay it several times. Compare state values, not just the appearance of the terminal. A different message history can produce the same final location; a renderer could also draw two different internal states identically. For this assignment, specify which state fields must agree and preserve the commands as data.

A canonical serialization puts fields in a stable order and uses a defined encoding. A digest of that serialization is a convenient compact comparison value. Equal digests are not a substitute for defining the state schema or proving correctness, and a process-dependent hash should not be treated as a portable saved identifier. The reference uses JSON with sorted keys and a SHA-256 digest for the lesson's small flat state.

Remove hidden inputs where practical

Wall-clock time, nondeterministic event order and unrecorded random choices can make a replay diverge. Our lesson intentionally avoids them. If randomness is added later, record enough information to reproduce its effects and pin the relevant environment or random-generation policy; a bare seed is not a universal promise across every implementation and future rule change.

For the terminal capstone, save/load commands are interface operations rather than movement commands. A successful load starts a new replay segment from the loaded initial state. The bounded replay buffer also starts a new segment when full. These choices prevent a saved command list from pretending it began at a state that no longer matches the actual session.

Measure a defined region

A benchmark should say what is timed. The supplied microbenchmark times 10,000 calls through the headless Python update loop by default, across seven trials after an untimed warm-up. It excludes terminal drawing and file I/O. It includes Python loop overhead, state validation, rule processing and message selection. That result is not whole-game frames per second.

The workload cycles through right, left, wait and restart so it does not spend almost every iteration doing a cheap terminal-state no-op. A benchmark that reaches victory on iteration 18 and then repeatedly returns unchanged state would measure mostly a different code path from active play. Representative workload design matters before interpreting the number.

Preserve raw samples and avoid invented conclusions

Use a monotonic performance timer such as `time.perf_counter` for elapsed intervals. Keep every trial duration, the actual interpreter and platform, the command cycle and final state. Report a median as a summary while retaining the samples. A single fast run can reflect noise, and a mean alone can hide variability.

Do not claim one machine is faster than another without comparable workloads and measurement methods. Do not invent a required microsecond threshold for this educational game. The learning outcome is a defensible experiment: define the work, verify the result, measure repeatedly and limit the conclusion to what was actually tested.

Worked example

Predict before running `examples/example_05_3.py`.

```
import hashlib
import json

first = {"x": 9, "y": 9, "won": True}
second = {"won": True, "y": 9, "x": 9}
first_text = json.dumps(first, sort_keys=True, separators=(",", ":"))
second_text = json.dumps(second, sort_keys=True, separators=(",", ":"))
first_digest = hashlib.sha256(first_text.encode("utf-8")).hexdigest()
second_digest = hashlib.sha256(second_text.encode("utf-8")).hexdigest()
print("same canonical text:", first_text == second_text)
print("same digest:", first_digest == second_digest)
print("canonical text:", first_text)
```

The dictionaries were written with different key orders but represent the same flat state. Sorting keys yields the same serialized text for this schema. The digest compares those defined bytes; it does not demonstrate that the movement leading to the state was correct. Keep independent expected transitions in the test suite.

Expected demonstration output

```
same canonical text: True
same digest: True
canonical text: {"won":true,"x":9,"y":9}
```

Practical work and pass evidence

Complete `labs/student/lab_05_3.py` after the Lesson 05.2 helper works. Replay an empty session, a twenty-command session and the known route to the goal. Compare both state fields and canonical digests across repeated runs. Preserve exact commands and rule assumptions.

Run `tools/benchmark_game.py` as a reference experiment, then adapt the same method to your own implementation without comparing unlike workloads. Keep the raw JSON report and state which operations are included. Pass by reproducing a session and explaining why the microbenchmark is not a whole-game FPS measurement.

Changed-case challenge

Benchmark a deliberately poor workload that spends almost all iterations in a terminal state. Compare its timed operation with the active-play cycle before comparing durations.

Companion source study

[S17] Python time library: perf_counter; [S11] Python json library; [S18] Python hashlib library. Read the sections relevant to the named concepts; record the section studied and one additional example.

Five-question lesson quiz

Answer independently. Core questions 1-3: at least 5/6. All five: at least 8/10, plus practical evidence.

1. What information does a replay need besides button names?
2. Why exclude printing from a simulation microbenchmark?
3. Does one matching replay prove determinism for all inputs?
4. Why can a replay with identical commands diverge after a map or rule change?
5. Why is a loop that reaches victory early and then performs thousands of no-op updates a misleading active-play benchmark?

Evidence to retain: predictions, code or worksheet, actual output, changed-case reasoning, quiz answers and corrections.

Lesson 05.4: Finishing, usability and the first portfolio artifact

Learning outcome: Deliver a tiny game another person can run and understand.

Finish a small system rather than expand an unfinished one

A completed first game has a clear starting state, understandable controls, valid and invalid input behavior, a reachable success condition, a failure condition where specified, restart and a reliable way to leave. It can be built or run from documented source, tested and explained. It does not need commercial art, networking, an account system or a large feature list.

Use the Gate A specification as the acceptance boundary. Freeze optional feature ideas in a separate list while repairing missing required behavior. Adding enemies or an editor does not compensate for a save that corrupts state or a restart that carries over old energy. Scope control is part of engineering, not a sign of insufficient ambition.

Test from a clean extraction

Create a release candidate in a fresh folder without your incidental workspace files. Follow the README exactly. Verify the interpreter check, launch command, controls, tests, save/load and replay behavior. A program that works only because your editor injected a path or an old file already exists is not reproducible from the package alone.

Do not bundle virtual environments, private credentials, machine-specific caches or unrelated documents. Include source, tests, required small data files, instructions and a clear list of known limitations. Keep generated saves in a separate workspace. Name the version and preserve a short change history connecting the release to the tested source.

Ask a tester to use the instructions without coaching

Give another person the release candidate and README. Ask them to start, move, encounter a blocked action, finish or lose, restart, quit and run the tests. Observe where they hesitate or fail, without quietly supplying the missing step. Record three concrete observations and fix at least one usability issue.

A useful observation describes behavior: “the tester typed up because the controls were not visible” is better than “the tester was confused.” A possible repair is showing controls at launch and making help available. Verify the change with another clean attempt. Do not describe one informal session as a comprehensive accessibility or usability study.

Accessibility begins with clear feedback

Use readable text, consistent controls and messages that distinguish rejection from success. Do not communicate a required event only by color or by a fleeting animation. The terminal reference uses explicit symbols and status text rather than depending on sound or color. That is a bounded design choice, not a guarantee that every terminal or assistive technology has been tested.

Record untested access needs honestly. A later game-production volume treats accessibility in greater depth. For this checkpoint, demonstrate that core instructions and outcomes are available as text and that the user can restart or quit without guessing hidden commands.

Submit evidence, not only the game

The Gate A handoff includes your source, a rule specification, README, at least twelve learner-written tests, a saved replay with independently expected final state, a two-bug journal, a usability record and a short technical explanation. The supplied reference code may be studied, but submitting it unchanged does not demonstrate your implementation skill.

A reviewer should ask for one live rule change: perhaps a different starting-energy limit or one additional obstacle. Predict affected tests first. Explain what remains invariant and how save/replay compatibility is handled. A change that silently reinterprets an old replay under new rules is not an acceptable shortcut.

State what completion means

Passing this volume means you can reason about, implement, test and hand off a small program with explicit state and behavior. It is not a claim that you are already a professional engine architect or that a degree has been awarded. Preserve the artifact as the baseline for future mathematics, native programming and systems work.

The next teaching volume in the program order is Mathematics for Games and Computing, Modules 06-10. Keep your current program small and working; later lessons can reuse it as a controlled place to investigate vectors, motion and measurement rather than replacing it with a much larger engine before you understand the foundations.

Worked example

Predict before running `examples/example_05_4.py`.

```
from pathlib import Path
import subprocess
import sys

root = Path(__file__).resolve().parents[1]
command = [sys.executable, str(root / "capstone" / "reference" / "play.py"),
           "--demo"]
result = subprocess.run(command, capture_output=True, text=True,
                        timeout=10, check=True)
print(result.stdout.splitlines()[-1])
print("Reference demonstration completed; your own Gate A remains required.")
```

This example runs the completed reference game through a known winning sequence. The final status line is an example of release behavior a reviewer can reproduce; running the same command directly also displays the board. It is not a student submission, and the label states that distinction. The subprocess uses the same Python interpreter as the wrapper rather than assuming an unrelated launcher name.

Expected demonstration output

```
Energy=7 Turns=18 Status=won
Reference demonstration completed; your own Gate A remains required.
```

Practical work and pass evidence

Complete `CAPSTONE_SPEC.md` and the five-module project requirements. Build your own game under `capstone/student/` or an explicitly documented workspace. Replace the intentionally failing starter test with at least twelve independent tests and add a meaningful design change rather than merely submitting the reference.

Ask a tester to follow your README without coaching, record three observations and repair at least one issue. Submit the release evidence and demonstrate a live rule variation. The supplied game checker tests the reference, not your project; use your own test command and independent review for Gate A.

Changed-case challenge

Prepare a handoff note explaining one known limitation, one tested failure path and one proposed next feature, each clearly separated from claims of completed work.

Companion source study

[S01] Harvard CS50 Introduction to Programming with Python; [S12] Python unittest library; [S13] Pro Git: Recording Changes to the Repository. Read the sections relevant to the named concepts; record the section studied and one additional example.

Five-question lesson quiz

Answer independently. Core questions 1-3: at least 5/6. All five: at least 8/10, plus practical evidence.

1. What belongs in a clean-start README?
2. Why should color not be the only important cue?
3. What evidence should accompany a claimed improvement?
4. The supplied reference game passes its tests. Does copying it unchanged satisfy the student Gate A implementation requirement?
5. A tester needs verbal instructions that are absent from the README. What should the release review record?

Evidence to retain: predictions, code or worksheet, actual output, changed-case reasoning, quiz answers and corrections.

Primary-source reading directory

These are companion sources for language/tool semantics, not copied textbook chapters. The lesson explanations, game rules, examples, quizzes and projects are original instructional material. Official documentation pages were consulted on September 28, 2026. The Python `/3/` pages are rolling documentation; use matching documentation for your installed interpreter. The package's executed environment is recorded separately in VALIDATION.md.

S01 - Harvard CS50 Introduction to Programming with Python

Beginner lecture companion: variables, conditions, loops, exceptions, tests and files.

Open official source

S02 - Python: An Informal Introduction

Numbers, text, assignments and introductory syntax.

Open official source

S03 - Python Setup and Usage

Use the installation and launcher instructions appropriate to the actual operating system.

Open official source

S04 - Python Built-in Types

Numeric, Boolean, sequence and mapping semantics. Reference reading, not a beginner lecture replacement.

Open official source

S05 - Python: Floating-Point Arithmetic

Representation limits and the distinction between displayed rounding and numerical value.

Open official source

S06 - Python: More Control Flow Tools

Conditions, range, loops, function definitions, arguments and return values.

Open official source

S07 - Python: Data Structures

Lists, dictionaries, copying and iteration.

Open official source

S08 - Python: Errors and Exceptions

Expected failures, specific exception handling and raising errors.

Open official source

S09 - Python: Classes

Names, objects, instances, methods and state ownership.

Open official source

S10 - Python: Modules

Import behavior, module namespaces and script entry points.

Open official source

S11 - Python json library

Encoding/decoding, duplicate-key hooks, resource cautions and JSON behavior.

Open official source

S12 - Python unittest library

Test cases, assertions, fixtures, discovery and runners.

Open official source

S13 - Pro Git: Recording Changes to the Repository

Working tree, staging, differences and commits.

Open official source

S14 - Python math library

Finite-value checks and numerical functions.

Open official source

S15 - Python os library: replace

File replacement semantics; do not infer universal power-loss durability.

Open official source

S16 - Git revert documentation

Recording a new commit that reverses an earlier change.

Open official source

S17 - Python time library: perf_counter

Performance-counter elapsed timing and its scope.

Open official source

S18 - Python hashlib library

Digest APIs used for comparison of defined serialized bytes.

Open official source